

SDET Interview Notes

QA Automation Engineer (3+ years) — prepared against the job description

Manual testing, Java, Selenium, TestNG, Cucumber, framework design, API and Postman, Playwright, CI/CD, JMeter, security testing, test strategy and scenarios

Ujjwal Tyagi | 9 October 2026

Contents

1. Read This First: JD Map and Your Introduction	3
2. Manual Testing and QA Fundamentals	5
3. Core Java	12
4. Selenium WebDriver	23
5. TestNG	31
6. Cucumber and BDD	34
7. Automation Framework Design	37
8. API Testing and Postman	40
9. Playwright (Basics)	47
10. Git, Maven, Jenkins, CI/CD and Docker	49
11. Performance Testing with JMeter (Basics)	53
12. Security Testing (Basics)	55
13. Test Strategy, Scenarios and Behavioural Questions	58
14. Last-Day Revision Sheet	62

1. Read This First: JD Map and Your Introduction

What this company is asking for

The JD is for a QA Automation Engineer who owns quality end to end: UI, API, performance and security. Product companies test three things in the interview: how well you code, how well you explain your framework, and how you think about quality.

JD requirement	Where it is in these notes	Depth you need
UI automation: Selenium and/or Playwright	Sections 4, 5, 6, 7, 9	Deep
API automation: Postman, REST, JSON, all HTTP methods	Section 8	Deep
Schema validation and negative testing	Section 8	Deep
Parameterized and data-driven testing	Sections 5, 6, 8	Deep
Regression suite ownership and maintenance	Sections 2, 7, 13	Deep
Test strategy, test planning, coverage	Sections 2, 13	Deep
Java scripting	Section 3	Deep
Git	Section 10	Medium
CI/CD with Jenkins (good to have)	Section 10	Medium
JMeter performance testing	Section 11	Basic, but know the terms
API interception, OWASP Top 10, ZAP	Section 12	Basic, but know the terms
Docker, cloud (good to have)	Section 10	Basic
Agile and Scrum	Section 2	Medium

Rules for answering

- Answer in three steps: a one-line definition, how you used it in your project, and one example.
- Say "in my framework" and "in my project" often. Interviewers trust experience more than definitions.
- If you do not know something, say: "I have not used it hands-on, but my understanding is..." and give the concept. Never bluff.
- For coding questions, speak while you write: approach first, then code, then edge cases.
- Be honest about depth. For JMeter and security, say you know the concepts and have basic exposure, and that you are keen to own them.

Your introduction (90 seconds)

Q1. Tell me about yourself.

Fill the brackets with your own facts and practise it aloud until it is smooth.

"I am Ujjwal Tyagi, a QA engineer with about [2.5] years of experience in manual and automation testing. In my current role at [company] I work on [domain, for example a healthcare or HR web application].

My main work is UI automation with Selenium and Java. I built and maintain a hybrid framework using TestNG, the Page Object Model, Maven, and Extent reports, with data-driven tests from Excel and BDD scenarios in Cucumber. The suite runs from Jenkins on every build, and I maintain about [number] regression test cases in it.

On the API side I test REST services with Postman: CRUD flows, status code and body assertions, environment variables, and negative tests. I have also worked with REST Assured, and I am now working with Playwright and TypeScript.

Apart from automation I do test planning, test case design, defect triage in Jira, and regression sign-off in an Agile team. I am looking for a role where I can own automation across UI and API and grow into performance and security testing, which is why this role interests me."

Interview tip: End your introduction on the JD. The last line tells the interviewer you read the role and want exactly that work.

Q2. Explain your current project and your role in it.

Use this order: what the product does (2 lines), the team and process (Agile, sprint length), your responsibilities (automation, API, regression, defect triage), and one achievement with a number, such as "reduced regression time from 2 days to 3 hours".

Q3. Why do you want to leave your current company? Why this company?

Keep it positive: "I have learned a lot, and now I want broader ownership. This role covers UI, API, performance and security in one place, and a product company lets me see the long-term impact of the quality work I do."

2. Manual Testing and QA Fundamentals

Basics

Q4. What is software testing, and why is it needed?

Testing is the process of checking that software does what the requirements say and finding defects before users do. It is needed because a defect found in production costs far more to fix than one found early, and because failures damage money, safety and reputation.

Q5. What is the difference between QA, QC and testing?

	QA	QC	Testing
Focus	The process	The product	Finding defects
Nature	Preventive	Detective	Detective
Example	Defining review and coding standards	Reviewing a build against requirements	Executing test cases

Q6. What is the difference between verification and validation?

Verification asks "are we building the product right?" It is static: reviews, walkthroughs and inspections of documents and code, without running the software. Validation asks "are we building the right product?" It is dynamic: running the software and checking the behaviour.

Q7. What is the difference between an error, a defect, a bug and a failure?

An error is a human mistake in code or a document. A defect is the flaw that mistake creates in the software. A bug is a defect found during testing and accepted by the developer. A failure is when the defect shows up while the software runs, usually in front of a user.

Q8. What are the seven principles of testing?

- Testing shows the presence of defects, not their absence.
- Exhaustive testing is impossible.
- Early testing saves time and money.
- Defects cluster together: a few modules hold most defects.
- Pesticide paradox: the same tests stop finding new bugs, so tests must be reviewed and updated.
- Testing is context dependent.
- Absence-of-errors fallacy: a bug-free product is useless if it does not meet user needs.

SDLC and STLC

Q9. What is SDLC? Name its phases.

The Software Development Life Cycle is the process for building software: requirement gathering, analysis, design, development, testing, deployment, and maintenance.

Q10. What is STLC? Explain each phase.

Phase	What happens	Output
Requirement analysis	Study requirements, find what is testable, raise queries	Query log, RTM draft
Test planning	Scope, approach, effort, tools, risks, schedule	Test plan
Test case design	Write scenarios, cases, and test data; review them	Test cases, test data, RTM
Environment setup	Prepare the test environment; run a smoke check	Ready environment
Test execution	Run cases, log defects, retest, run regression	Execution report, defect reports
Test closure	Check exit criteria, metrics, lessons learned	Test summary report

Q11. What are entry and exit criteria?

Entry criteria are conditions that must be true before testing starts: an approved test plan, a ready environment, a build that passes smoke testing. Exit criteria are conditions for stopping: all planned cases executed, no open critical or high defects, the agreed pass percentage reached, and regression passed.

Q12. Explain the Waterfall model, V-model and Agile.

Waterfall runs the phases one after another; testing comes late and change is costly. In the V-model every development phase has a matching test phase (requirements with acceptance testing, design with system and integration testing, coding with unit testing), so test planning starts early. Agile builds the product in short iterations called sprints, with testing in every sprint and frequent feedback.

Agile and Scrum

Q13. What is Agile? What is Scrum?

Agile is an iterative way of working that delivers small, working pieces of software often and welcomes change. Scrum is the most used Agile framework. Work is done in sprints of 2 to 4 weeks.

Q14. What are the Scrum roles, ceremonies and artifacts?

Type	Items
Roles	Product Owner (owns the backlog and priorities), Scrum Master (removes blockers, guards the process), Development Team (developers and testers)
Ceremonies	Sprint planning, daily stand-up, backlog refinement (grooming), sprint review (demo), sprint retrospective
Artifacts	Product backlog, sprint backlog, increment

Q15. What do you do as a tester in a sprint?

In refinement I review stories and acceptance criteria and raise questions. In planning I estimate testing effort. During the sprint I write test cases, test stories as they are delivered, log and retest defects, and automate the stable cases. Before the sprint ends I run regression and give sign-off. In the retrospective I raise process improvements.

Q16. What is a user story, an epic, acceptance criteria, Definition of Ready and Definition of Done?

A user story is a small requirement written as "As a [user], I want [goal], so that [benefit]". An epic is a large body of work split into stories. Acceptance criteria are the conditions the story must meet to be accepted. Definition of Ready says when a story can be picked up. Definition of Done says when it is complete: coded, reviewed, tested, defects closed, automated where agreed.

Q17. What are story points, velocity and a burndown chart?

Story points are a relative estimate of effort and complexity. Velocity is the number of story points a team completes per sprint. A burndown chart shows remaining work against days left in the sprint.

Levels and types of testing

Q18. What are the levels of testing?

Unit testing (developers test one function or class), integration testing (modules working together), system testing (the complete application against requirements), and user acceptance testing (business users confirm it meets their needs).

Q19. What is the difference between functional and non-functional testing?

Functional testing checks what the system does: features and business rules. Non-functional testing checks how well it does it: performance, security, usability, compatibility, reliability.

Q20. What is the difference between smoke and sanity testing?

	Smoke	Sanity
Purpose	Is the build stable enough to test?	Does a specific fix or change work?
Scope	Wide and shallow: critical flows	Narrow and deep: one area
When	On every new build	After a small change or bug fix
Scripted	Usually yes, often automated	Usually not

Q21. What is the difference between retesting and regression testing?

Retesting runs the failed test again to confirm that a specific defect is fixed. Regression testing runs existing tests to confirm that a change has not broken features that worked before. Retesting is planned around defects; regression is planned around impact, and is the main candidate for automation.

Q22. How do you select test cases for regression?

- Cases for the changed feature and the areas that depend on it (impact analysis).
- Core business flows such as login, checkout, payment.
- Areas with many past defects.
- Cases for recently fixed defects.
- Integration points between modules.

When time is short I run a risk-based subset: priority 1 flows first, then impacted areas.

Q23. What is the difference between black-box, white-box and grey-box testing?

Black-box tests behaviour without knowing the code. White-box tests the internal logic and code paths. Grey-box combines them: the tester knows some internals, such as the database or API, and uses that to design better tests.

Q24. Explain alpha, beta and UAT.

Alpha testing is done in-house before release. Beta testing is done by real users in their own environment before the general release. UAT is done by the client or business users to accept the product.

Q25. What is exploratory testing? What is ad hoc testing?

Exploratory testing is learning, designing and executing tests at the same time, guided by a goal or charter, with notes on what was covered. Ad hoc testing is informal testing with no plan or documentation, aimed at breaking the system.

Q26. Define these quickly: integration approaches, compatibility, usability, localization, end-to-end, monkey testing.

- Integration approaches: big bang (all at once), top-down (uses stubs), bottom-up (uses drivers), sandwich (both).
- Compatibility: works across browsers, devices, OS versions.
- Usability: easy and clear for users.
- Localization and globalization: works for a specific locale, and is built to support many.
- End-to-end: a full business flow across systems.
- Monkey testing: random inputs to see whether the system crashes.

Q27. What are stubs and drivers?

Both are dummy programs used in integration testing. A stub replaces a lower-level module that is not ready and is called by the module under test. A driver replaces a higher-level module and calls the module under test.

Test design techniques

Q28. What is equivalence partitioning? Give an example.

Inputs are divided into groups that should behave the same way, and one value is tested from each group. For an age field that accepts 18 to 60: one invalid value below 18, one valid value between 18 and 60, one invalid value above 60.

Q29. What is boundary value analysis? Give an example.

Defects gather at the edges of ranges, so we test the boundaries. For 18 to 60: 17, 18, 19 and 59, 60, 61.

Q30. What is a decision table? What is state transition testing?

A decision table lists combinations of conditions and the expected action for each, used when business rules depend on several inputs. State transition testing checks how the system moves between states for given events, for example an account that locks after three wrong passwords.

Q31. What are error guessing and use case testing?

Error guessing uses experience to guess where defects are likely, such as empty fields, special characters and very long input. Use case testing derives tests from user flows: the main flow and its alternate and exception flows.

Test artifacts

Q32. What is the difference between a test scenario and a test case?

A test scenario is a one-line statement of what to test, such as "verify login functionality". A test case is a detailed set of steps, data and expected results for one condition of that scenario.

Q33. What does a test case contain?

Test case ID, title, module, preconditions, test steps, test data, expected result, actual result, status, priority, and the requirement ID it traces to.

Q34. What makes a good test case?

It is clear, has one objective, is independent of other cases, is repeatable, has a precise expected result, traces to a requirement, and covers positive, negative and boundary conditions.

Q35. What is a test plan? What does it contain?

A test plan is the document that describes the scope, approach, resources and schedule of testing. It contains: objectives, scope (in and out), test approach, test levels and types, entry and exit criteria, environment, tools, roles, schedule, deliverables, risks and mitigation, and defect management.

Q36. What is the difference between a test plan and a test strategy?

A test strategy is a high-level document, usually at organisation or product level, that sets the overall approach and standards and rarely changes. A test plan is project or release specific and says what will be tested, by whom, when and how.

Q37. What is an RTM?

A Requirement Traceability Matrix maps each requirement to its test cases and defects. It proves that every requirement is covered and shows the impact when a requirement changes.

Q38. What is test coverage, and how do you measure it?

Coverage is how much of the product the tests exercise. I measure requirement coverage through the RTM (requirements with at least one test), and automation coverage as automated cases divided by automatable cases. Code coverage is measured by developers with tools such as JaCoCo.

Defect management

Q39. Explain the defect life cycle.

New, Assigned, Open, Fixed, Ready for Retest, Retest, then Verified and Closed. If the fix fails it becomes Reopened. Other states are Rejected (not a defect), Duplicate, Deferred (fix in a later release), and Not Reproducible.

Q40. What is the difference between severity and priority? Give examples.

Severity is the impact of the defect on the system, set by the tester. Priority is how soon it must be fixed, set by the product owner or manager.

Case	Example
High severity, high priority	Payment fails for all users
High severity, low priority	The app crashes on a rarely used legacy report
Low severity, high priority	The company logo or name is wrong on the home page
Low severity, low priority	A spelling mistake on a rarely visited help page

Q41. What do you include in a good bug report?

A clear summary, environment and build, preconditions, exact steps to reproduce, test data, expected and actual result, severity and priority, and evidence: screenshots, video, logs, and for API defects the request and response.

Q42. A developer says "it is not a bug" or "it works on my machine". What do you do?

I stay factual. I recheck the requirement and reproduce the issue, then share exact steps, data, environment, logs and a recording. I compare environments and builds with the developer. If we still disagree on expected behaviour, I take it to the product owner, because the requirement decides, not opinion.

Q43. What is defect leakage? What is defect density? What is defect removal efficiency?

Defect leakage is defects that escaped to a later stage or production, measured as defects found after release divided by total defects. Defect density is defects per size of the module, such as per thousand lines of code or per story. Defect removal efficiency is defects found before release divided by all defects, as a percentage.

Q44. What do you do if you find a defect in production?

I reproduce it and assess the impact, log it with full evidence, and inform the team at once. After the fix I retest and run regression around it. Then I do a root cause analysis on why it escaped (missing case, environment gap, data gap) and add a test, preferably automated, so it cannot escape again.

Q45. What is root cause analysis?

RCA finds why a defect happened and why it was not caught, so the cause can be removed. Common techniques are the 5 Whys and the fishbone diagram.

Practical questions

Q46. The requirements are unclear or missing. How do you test?

I ask the product owner or business analyst and record the answers. Meanwhile I learn from the existing application, similar features, old test cases and defects, and I do exploratory testing. I write down my assumptions and get them confirmed.

Q47. You have very little time before release. What do you test?

I use risk-based testing: smoke tests first, then the changed features, then the critical business flows and high-risk areas. I run the automated regression in parallel, state clearly what was not tested, and let the team make an informed release decision.

Q48. When do you stop testing?

When the exit criteria are met: planned cases executed, critical and high defects closed, regression passed, coverage achieved, and remaining risk accepted by stakeholders. Deadlines and budget also apply, with the risk made visible.

Q49. Write test scenarios for a login page.

- Functional: valid login, invalid password, invalid username, both empty, case sensitivity, trimmed spaces, "remember me", "forgot password", logout, and back button after logout.
- Security: account lock after repeated failures, password masked, no password in the URL or logs, SQL injection and script input rejected, session timeout, HTTPS.

- UI and usability: field labels, tab order, Enter key submits, error messages clear and not revealing which field was wrong.
- Non-functional: response time, multiple browsers, concurrent logins.

Q50. Write test scenarios for an ATM, a lift, or a pen.

Use the same structure every time, and say the structure aloud: functional, negative, boundary, usability, performance, security, and compatibility. For an ATM: valid and invalid PIN, card block after three wrong PINs, withdrawal within and above balance and daily limit, denominations, receipt, session timeout, card retained on timeout, network failure during a transaction.

Q51. Which test metrics do you report?

Test execution progress, pass and fail percentage, defect counts by severity and status, defect leakage, defect reopen rate, requirement coverage, automation coverage, and regression execution time.

3. Core Java

Java basics

Q52. What are the main features of Java?

Object-oriented, platform independent ("write once, run anywhere"), robust (strong type checking, exception handling, automatic memory management), secure, multithreaded, and high performance through the JIT compiler.

Q53. What is the difference between JDK, JRE and JVM?

	What it is	Contains
JVM	Runs bytecode; it is platform dependent	Class loader, memory areas, execution engine
JRE	Environment to run Java programs	JVM + core libraries
JDK	Kit to develop Java programs	JRE + compiler (javac), debugger, tools

Q54. Why is Java platform independent?

The compiler turns source code into bytecode (.class), not machine code. Any machine with a JVM can run that bytecode. The bytecode is platform independent; the JVM is platform specific.

Q55. Explain `public static void main(String[] args)`.

`public` so the JVM can call it from anywhere. `static` so the JVM can call it without creating an object. `void` because it returns nothing. `main` is the name the JVM looks for. `String[] args` holds command-line arguments.

Q56. What are the data types in Java?

Primitive types: byte (1 byte), short (2), int (4), long (8), float (4), double (8), char (2), boolean.
Non-primitive (reference) types: String, arrays, classes, interfaces.

Q57. What are the types of variables?

Local variables are declared inside a method, live on the stack, and have no default value. Instance variables are declared in a class, belong to each object, and live on the heap. Static variables are declared with `static`, belong to the class, and are shared by all objects.

Q58. What is type casting?

Widening (implicit) converts a smaller type to a larger one, such as `int` to `long`. Narrowing (explicit) converts a larger type to a smaller one and needs a cast, such as `int i = (int) 10.5;`, and may lose data. For objects, upcasting is child to parent (automatic) and downcasting is parent to child (explicit).

Q59. What is the difference between `==` and `equals()`?

`==` compares references for objects (whether both point to the same object) and values for primitives. `equals()` compares content, when the class overrides it, as `String` does.

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);           // false: two different objects
System.out.println(a.equals(b));      // true: same content
```

Q60. What are wrapper classes, autoboxing and unboxing?

Wrapper classes turn primitives into objects: Integer, Double, Character, Boolean. Collections need them because collections store objects only. Autoboxing is the automatic conversion of a primitive to its wrapper; unboxing is the reverse.

Q61. What do `break` and `continue` do?

`break` exits the loop. `continue` skips the rest of the current iteration and moves to the next.

OOP concepts

Q62. What are the four pillars of OOP? Where did you use each in your framework?

Pillar	Meaning	In a Selenium framework
Encapsulation	Hide data, expose it through methods	Page classes: private By locators or WebElements, public action methods
Inheritance	A child class reuses a parent class	Test classes extend <code>BaseTest</code> ; page classes extend <code>BasePage</code>
Polymorphism	One name, many forms	<code>WebDriver driver = new ChromeDriver();</code> and overloaded utility methods
Abstraction	Show what, hide how	<code>WebDriver</code> is an interface; abstract <code>BasePage</code> with common methods

Interview tip: This is asked in almost every SDET interview. Always answer with the framework example, not only the definition.

Q63. What is a class and what is an object?

A class is a blueprint that defines variables and methods. An object is an instance of a class, created with `new`, and it occupies memory on the heap.

Q64. What is a constructor? What are its types?

A constructor is a special block that runs when an object is created and initialises it. It has the same name as the class and no return type. Types: default (no arguments, supplied by the compiler if you write none) and parameterized. Constructors can be overloaded but not overridden.

Q65. What is constructor chaining?

One constructor calling another: `this()` calls a constructor of the same class, and `super()` calls the parent's constructor. Either must be the first statement.

Q66. What is the difference between `this` and `super`?

`this` refers to the current object: its variables, methods and constructors. `super` refers to the parent class: its variables, methods and constructor.

Q67. What is inheritance? What types does Java support?

Inheritance lets a class acquire the properties and methods of another using `extends`. Java supports single, multilevel and hierarchical inheritance with classes. Multiple inheritance with classes is not supported, to avoid the diamond problem (ambiguity when two parents have the same method). It is achieved through interfaces.

Q68. What is the difference between method overloading and method overriding?

	Overloading	Overriding
Where	Same class	Parent and child class
Signature	Same name, different parameters	Same name, same parameters
Return type	Can differ	Same or a subtype (covariant)
Binding	Compile time (static polymorphism)	Runtime (dynamic polymorphism)
Access	Any	Cannot be more restrictive
static, final, private methods	Can be overloaded	Cannot be overridden

Examples in Selenium: `findElement` is overridden by each driver class. In utilities, `waitForElement(By)` and `waitForElement(By, int)` are overloaded.

Q69. Can you override a static method? Can you overload `main`?

A static method cannot be overridden. If the child declares the same static method, it is method hiding. `main` can be overloaded, but the JVM calls only the one with `String[] args`.

Q70. What is the difference between an abstract class and an interface?

	Abstract class	Interface
Methods	Abstract and concrete	Abstract by default; default and static methods since Java 8; private since Java 9
Variables	Any kind	<code>public static final</code> only
Constructor	Yes	No
Inheritance	A class extends one	A class implements many
Use when	Classes share state and code	Unrelated classes share a contract

Neither can be instantiated.

Q71. What is encapsulation? How is it different from abstraction?

Encapsulation wraps data and methods together and hides the data with `private`, exposing it through getters and setters. Abstraction hides implementation and shows only the functionality. Encapsulation is about protecting data; abstraction is about hiding complexity.

Q72. What are access modifiers?

Modifier	Same class	Same package	Subclass (other package)	Everywhere
<code>private</code>	Yes	No	No	No
<code>default</code>	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

Q73. What does `static` mean?

A static member belongs to the class, not to an object. Static variables are shared by all objects. Static methods can be called by class name and cannot use `this` or non-static members directly. A static block runs once when the class loads.

Q74. What is the difference between `final`, `finally` and `finalize`?

`final` is a keyword: a final variable cannot be reassigned, a final method cannot be overridden, a final class cannot be extended. `finally` is a block that always runs after try-catch, used for cleanup. `finalize()` was a method called by the garbage collector before destroying an object; it is deprecated.

String

Q75. Why is `String` immutable?

Once created, a `String` object cannot change; any change creates a new object. Reasons: the `String` pool can share objects safely, strings are safe to use across threads, they are safe as `HashMap` keys because the hash code never changes, and it protects values such as file paths and URLs.

Q76. What is the `String` constant pool?

A special area in the heap that stores string literals. `String a = "java";` reuses an existing pooled object if one exists. `new String("java")` always creates a new object on the heap.

Q77. What is the difference between `String`, `StringBuilder` and `StringBuffer`?

	<code>String</code>	<code>StringBuilder</code>	<code>StringBuffer</code>
Mutable	No	Yes	Yes
Thread safe	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for many changes	Fastest	Slower than <code>StringBuilder</code>

Q78. Name the `String` methods you use most.

`length()`, `charAt()`, `substring()`, `indexOf()`, `contains()`, `equals()`, `equalsIgnoreCase()`, `trim()`, `split()`, `replace()`, `toUpperCase()`, `toLowerCase()`, `toCharArray()`, `isEmpty()`, `startsWith()`, `endsWith()`, `String.valueOf()`.

Exception handling

Q79. What is an exception? Explain the hierarchy.

An exception is an event that disrupts the normal flow of a program. `Throwable` is the root, with two children: `Error` (serious problems such as `OutOfMemoryError`, not meant to be handled) and `Exception`. Under `Exception`, `RuntimeException` and its children are unchecked; all others are checked.

Q80. What is the difference between checked and unchecked exceptions?

Checked exceptions are checked at compile time and must be handled or declared: `IOException`, `FileNotFoundException`, `SQLException`, `InterruptedException`. Unchecked exceptions occur at runtime: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `ClassCastException`, `NumberFormatException`.

Q81. What is the difference between `throw` and `throws`?

`throw` is used inside a method to throw one exception object explicitly. `throws` is used in the method signature to declare exceptions the method may pass to its caller.

Q82. Explain try, catch and finally. Does finally always run?

Risky code goes in try. catch handles the exception. finally runs whether or not an exception occurs, and is used to close resources. It does not run only if the JVM exits, for example on System.exit(). A try can have many catch blocks, ordered from specific to general.

Q83. What is try-with-resources?

A try that declares resources in brackets; they are closed automatically. The resource must implement AutoCloseable.

```
try (FileInputStream fis = new FileInputStream("data.xlsx")) {  
    // use fis  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Q84. How do you create a custom exception?

Extend Exception for a checked one or RuntimeException for an unchecked one.

```
public class FrameworkException extends RuntimeException {  
    public FrameworkException(String message) {  
        super(message);  
    }  
}
```

Collections

Q85. What is the Collections framework? Explain the hierarchy.

A set of interfaces and classes for storing and handling groups of objects. Collection has three main child interfaces: List, Set and Queue. Map is separate and stores key-value pairs.

Interface	Classes	Nature
List	ArrayList, LinkedList, Vector	Ordered, allows duplicates, index based
Set	HashSet, LinkedHashSet, TreeSet	No duplicates
Queue	PriorityQueue, ArrayDeque	Usually first in, first out
Map	HashMap, LinkedHashMap, TreeMap, Hashtable	Key-value pairs, unique keys

Q86. What is the difference between an array and an ArrayList?

An array has a fixed size and can hold primitives and objects. An ArrayList grows dynamically, holds objects only, and offers ready methods such as add, remove and contains.

Q87. What is the difference between ArrayList and LinkedList?

ArrayList uses a dynamic array: fast random access by index, slow insert and delete in the middle because elements shift. LinkedList uses a doubly linked list: fast insert and delete, slow access because it traverses nodes.

Q88. What is the difference between List and Set?

A List keeps insertion order and allows duplicates and index access. A Set does not allow duplicates and has no index.

Q89. What is the difference between HashSet, LinkedHashSet and TreeSet?

HashSet has no order and allows one null. LinkedHashSet keeps insertion order. TreeSet keeps elements sorted and does not allow null.

Q90. What is the difference between HashMap, LinkedHashMap and TreeMap?

HashMap has no order and allows one null key. LinkedHashMap keeps insertion order. TreeMap sorts by key and does not allow a null key.

Q91. What is the difference between HashMap and Hashtable?

HashMap is not synchronized, is faster, and allows one null key and many null values. Hashtable is synchronized, slower, and allows no nulls. For thread safety today we use ConcurrentHashMap.

Q92. How does HashMap work internally?

It stores entries in an array of buckets. On put, it calls hashCode() on the key to find the bucket index. If the bucket is empty, the entry goes in. If not, that is a collision: it compares keys with equals(); an equal key means the value is replaced, otherwise the new entry is added to that bucket's linked list. From Java 8, a bucket with more than 8 entries becomes a balanced tree. The default capacity is 16 and the load factor is 0.75; when it fills past that, the map resizes.

Q93. Why must equals() and hashCode() be overridden together?

The contract says equal objects must have equal hash codes. If only equals() is overridden, two equal keys may land in different buckets and a HashMap or HashSet will treat them as different.

Q94. What is the difference between Comparable and Comparator?

Comparable gives a class one natural order through compareTo(), written inside the class. Comparator defines extra orders through compare(), written outside the class, so you can sort the same objects in many ways.

Q95. What is the difference between Iterator and ListIterator? What is fail-fast?

An Iterator moves forward only and works on all collections. A ListIterator works on lists and moves both ways. Fail-fast iterators throw ConcurrentModificationException if the collection is changed while iterating; use the iterator's own remove() instead.

Q96. Where did you use collections in your framework?

- List<WebElement> from findElements.
- Set<String> from getWindowHandles.
- Map<String, String> to hold a row of test data from Excel or a DataTable.
- HashMap for request headers and for config values.
- ArrayList to collect UI values before comparing them with expected data.

Other important topics

Q97. Where do objects and variables live in memory? What is garbage collection?

Objects and instance variables live on the heap. Method calls and local variables live on the stack, one per thread. Garbage collection automatically frees heap objects that have no references. System.gc() only requests it.

Q98. What is multithreading? How do you create a thread?

Multithreading runs several threads at the same time. Create a thread by extending Thread or by implementing Runnable, which is preferred because the class can still extend another. start() creates a new thread; calling run() directly does not.

Q99. What is synchronized? What is ThreadLocal, and why does it matter in automation?

synchronized lets only one thread at a time run a method or block. ThreadLocal gives each thread its own copy of a variable. In parallel test execution we store the WebDriver in a ThreadLocal<WebDriver> so each test thread uses its own browser and tests do not interfere.

Q100. What is the difference between sleep() and wait()?

sleep() belongs to Thread, pauses the thread for a fixed time, and keeps any lock. wait() belongs to Object, must be called inside a synchronized block, releases the lock, and resumes on notify().

Q101. What is the Singleton pattern? Write one.

A class that allows only one object. Used for the driver factory or config reader.

```
public class ConfigReader {
    private static ConfigReader instance;
    private ConfigReader() { } // private constructor
    public static synchronized ConfigReader getInstance() {
        if (instance == null) {
            instance = new ConfigReader();
        }
        return instance;
    }
}
```

Q102. What are the main Java 8 features?

Lambda expressions, functional interfaces, the Stream API, method references, default and static methods in interfaces, Optional, and the new date and time API.

Q103. What is a lambda expression? What is a functional interface?

A functional interface has exactly one abstract method, such as Runnable, Comparator, Predicate, Function, Consumer, Supplier. A lambda is a short way to implement one: (a, b) -> a + b.

Q104. What is the Stream API? Give examples.

A stream processes a collection in a pipeline of operations without changing the source. Intermediate operations (filter, map, sorted, distinct) are lazy; terminal operations (collect, forEach, count) produce the result.

```
List<Integer> even = nums.stream().filter(n -> n % 2 == 0).collect(Collectors.toList());

// Selenium: collect the text of all links
List<String> names = driver.findElements(By.tagName("a")).stream()
    .map(WebElement::getText)
    .filter(t -> !t.isEmpty())
    .collect(Collectors.toList());
```

Q105. What is the difference between Optional, null checks, and why does it exist?

Optional is a container that may or may not hold a value. It makes a missing value explicit and helps avoid NullPointerException: Optional.ofNullable(x).orElse("default").

Q106. What are serialization and the transient keyword?

Serialization converts an object into a byte stream so it can be saved or sent; the class must implement Serializable. A transient field is skipped. In API automation, converting a POJO to JSON is also called serialization, and JSON to POJO is deserialization.

Q107. What is the difference between a shallow copy and a deep copy?

A shallow copy copies the object but shares the nested objects. A deep copy copies the object and all nested objects, so the two are independent.

Q108. Is Java pass by value or pass by reference?

Always pass by value. For objects, the value passed is a copy of the reference, so the method can change the object's state but cannot make the caller's variable point to a new object.

Java coding questions

Product companies almost always ask you to write code. Every program below was compiled and run. Learn the approach, then practise writing each one by hand without looking.

Interview tip: Before writing, say the approach in one line and ask about edge cases: null or empty input, case sensitivity, spaces, negative numbers. Then give the time complexity at the end.

Q109. Reverse a string.

```
static String reverse(String s) {
    String rev = "";
    for (int i = s.length() - 1; i >= 0; i--) {
        rev = rev + s.charAt(i);
    }
    return rev;
}
// reverse("selenium") -> "muineles"
// One line: new StringBuilder(s).reverse().toString()
```

Q110. Check whether a string is a palindrome.

```
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}
// "madam" -> true, "java" -> false. Two pointers, O(n).
```

Q111. Count the occurrences of each character in a string.

```
static Map<Character, Integer> charCount(String s) {
    Map<Character, Integer> map = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }
    return map;
}
// "automation" -> {a=2, u=1, t=2, o=2, m=1, i=1, n=1}
// To print only duplicates, print the entries whose value is greater than 1.
```

Q112. Find the first non-repeated character.

```
static char firstNonRepeated(String s) {
    Map<Character, Integer> map = charCount(s); // LinkedHashMap keeps order
    for (Map.Entry<Character, Integer> e : map.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }
    return '-';
}
// "swiss" -> 'w'
```

Q113. Remove duplicate characters from a string.

```
static String removeDuplicates(String s) {
    Set<Character> set = new LinkedHashSet<>();
    for (char c : s.toCharArray()) set.add(c);
    StringBuilder sb = new StringBuilder();
    for (char c : set) sb.append(c);
    return sb.toString();
}
// "programming" -> "progamin"
```

Q114. Check whether two strings are anagrams.

```
static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] x = a.toLowerCase().toCharArray();
    char[] y = b.toLowerCase().toCharArray();
    Arrays.sort(x);
    Arrays.sort(y);
    return Arrays.equals(x, y);
}
// "Listen", "Silent" -> true
```

Q115. Reverse the words of a sentence.

```
static String reverseWords(String s) {
    String[] w = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = w.length - 1; i >= 0; i--) {
        sb.append(w[i]).append(i > 0 ? " " : "");
    }
    return sb.toString();
}
// "I love Java" -> "Java love I"
```

Q116. Count vowels and consonants.

```
static int[] countVowels(String s) {
    int v = 0, c = 0;
    for (char ch : s.toLowerCase().toCharArray()) {
        if ("aeiou".indexOf(ch) >= 0) v++;
        else if (ch >= 'a' && ch <= 'z') c++;
    }
    return new int[]{v, c};
}
// "Automation Testing" -> 8 vowels, 9 consonants
```

Q117. Separate the digits and letters of a string.

```
String s = "ab12cd3";
String digits = s.replaceAll("[^0-9]", ""); // "123"
String letters = s.replaceAll("[0-9]", ""); // "abcd"
```

Q118. Swap two numbers without a third variable.

```
int p = 5, q = 7;
p = p + q; // 12
q = p - q; // 5
p = p - q; // 7
```

Q119. Print the Fibonacci series.

```
static void fibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        System.out.print(a + " ");
        int c = a + b;
        a = b;
        b = c;
    }
}
// fibonacci(8) -> 0 1 1 2 3 5 8 13
```

Q120. Check whether a number is prime.

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
// Checking up to the square root is enough: O(sqrt n)
```

Q121. Find the factorial of a number, and reverse a number.

```
static long factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);    // 5 -> 120
}

static int reverseNumber(int n) {
    int rev = 0;
    while (n != 0) {
        rev = rev * 10 + n % 10;
        n = n / 10;
    }
    return rev;    // 1234 -> 4321
}
```

Q122. Check whether a number is an Armstrong number.

```
static boolean isArmstrong(int n) {
    int digits = String.valueOf(n).length(), sum = 0, t = n;
    while (t != 0) {
        sum += (int) Math.pow(t % 10, digits);
        t /= 10;
    }
    return sum == n;    // 153 -> true
}
```

Q123. Find the second largest number in an array.

```
static int secondLargest(int[] a) {
    int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int n : a) {
        if (n > first) {
            second = first;
            first = n;
        } else if (n > second && n != first) {
            second = n;
        }
    }
    return second;
}
// {5, 9, 2, 9, 7} -> 7. One pass, O(n), handles duplicates.
```

Q124. Sort an array without using a built-in method (bubble sort).

```
static void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
}
// {5, 1, 4, 2} -> {1, 2, 4, 5}. O(n squared).
```

Q125. Find the missing number in an array of 1 to n.

```
static int missingNumber(int[] a, int n) {
    int sum = n * (n + 1) / 2;
    for (int x : a) sum -= x;
    return sum;
}
// {1, 2, 4, 5}, n = 5 -> 3
```

Q126. Find the duplicate elements in an array.

```
static Set<Integer> duplicates(int[] a) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dup = new LinkedHashSet<>();
    for (int n : a) {
        if (!seen.add(n)) dup.add(n); // add() returns false if already present
    }
    return dup;
}
// {1, 2, 3, 2, 1, 5} -> [2, 1]
```

Q127. Solve these with Java 8 streams.

```
List<Integer> nums = Arrays.asList(4, 7, 2, 9, 4, 7);

// remove duplicates and sort in descending order -> [9, 7, 4, 2]
nums.stream().distinct().sorted(Comparator.reverseOrder()).collect(Collectors.toList());

// maximum -> 9
nums.stream().mapToInt(Integer::intValue).max().getAsInt();

// character frequency -> {a=2, u=1, t=2, o=2, m=1, i=1, n=1}
Map<Character, Long> freq = "automation".chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()));
```

Q128. How do you iterate over a HashMap?

```
for (Map.Entry<String, String> e : map.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
map.forEach((k, v) -> System.out.println(k + " = " + v)); // Java 8
```

4. Selenium WebDriver

Basics and architecture

Q129. What is Selenium? What are its components?

Selenium is an open-source suite for automating web browsers. Components: Selenium IDE (record and playback), Selenium WebDriver (the API that drives browsers through code), and Selenium Grid (runs tests in parallel on many machines and browsers). Selenium RC is the old, retired component.

Q130. What are the advantages and limitations of Selenium?

Advantages: free, supports many languages (Java, Python, C#, JavaScript), browsers and operating systems, and has a large community and integrations (TestNG, Maven, Jenkins). Limitations: web applications only, no built-in reporting, no built-in support for captcha, OTP, barcodes, images or desktop pop-ups, and no official vendor support.

Q131. Explain the Selenium WebDriver architecture.

There are four parts: the client library (your Java code), the protocol, the browser driver (chromedriver, geckodriver), and the browser. Each command in the script is converted into an HTTP request and sent to the browser driver, which is a small HTTP server. The driver passes the command to the browser natively and returns the response to the script.

In Selenium 3 the protocol was the JSON Wire Protocol, and commands were translated before reaching the browser. Selenium 4 uses the W3C WebDriver protocol, which browsers understand directly, so execution is more stable.

Q132. What is new in Selenium 4?

- W3C WebDriver protocol as the standard.
- Selenium Manager (from 4.6): downloads and manages the right driver automatically, so `System.setProperty` and `WebDriverManager` are no longer needed.
- Relative locators: `above`, `below`, `toLeftOf`, `toRightOf`, `near`.
- `driver.switchTo().newWindow(WindowType.TAB)` to open a new tab or window.
- Element-level screenshots.
- Waits take `Duration` in place of a number and a time unit.
- Chrome DevTools Protocol and BiDi support: network interception, console logs, geolocation.
- A redesigned Grid with Docker support.

Q133. Explain `WebDriver driver = new ChromeDriver();`

`WebDriver` is an interface and `ChromeDriver` is a class that implements it (through `RemoteWebDriver`). We store the object in a reference of the interface type. This is upcasting and runtime polymorphism: the same driver variable can hold `FirefoxDriver` or `EdgeDriver`, so tests are not tied to one browser.

Q134. What is the class hierarchy of WebDriver?

`SearchContext` (interface, declares `findElement` and `findElements`) is extended by `WebDriver` (interface). `RemoteWebDriver` (class) implements `WebDriver`, `JavaScriptExecutor` and `TakesScreenshot`. `ChromiumDriver` extends `RemoteWebDriver`, and `ChromeDriver` and `EdgeDriver` extend `ChromiumDriver`. `FirefoxDriver` extends `RemoteWebDriver`.

Locators and XPath

Q135. What locators does Selenium support? Which do you prefer?

Eight: id, name, className, tagName, linkText, partialLinkText, cssSelector, xpath. My order of preference is id, then name, then CSS selector, then XPath. id is fastest and most stable when it is unique and static. I use XPath when I need text or to move up to a parent.

Q136. What is the difference between absolute and relative XPath?

Absolute XPath starts from the root with a single slash (/html/body/div[2]/input) and breaks when the page structure changes. Relative XPath starts anywhere with a double slash (//input[@id='email']) and is more stable. Always use relative XPath.

Q137. Write the common XPath patterns.

//tag[@attribute='value']	basic
//input[@id='user' and @type='text']	two conditions
//button[text()='Login']	exact text
//a[contains(text(),'Sign')]	partial text
//input[contains(@id,'user_')]	partial attribute (dynamic ids)
//input[starts-with(@id,'user')]	starts with
//button[normalize-space()='Login']	ignores extra spaces
(//input[@type='checkbox'])[2]	second match
//table[@id='t1']//tr[3]/td[2]	row 3, column 2
//td[text()='Ujjwal']/following-sibling::td	cell next to a known cell

Q138. What are XPath axes?

Axes move from one node to related nodes:

Axis	Selects
parent::	The parent
child::	Direct children
ancestor::	All parents up to the root
descendant::	All children at any depth
following-sibling::	Siblings after the node
preceding-sibling::	Siblings before the node
following::	Everything after the node in the document
preceding::	Everything before the node

Example: //label[text()='Email']/following-sibling::input.

Q139. What is the difference between XPath and CSS selector?

CSS is usually faster and shorter, and it only moves forward (parent to child). XPath can move both ways (child to parent), and can locate by text. CSS cannot locate by visible text.

#id	.class	input[name='q']	div > input (direct child)
div input (descendant)		input[id^='user'] (starts)	
input[id\$='name'] (ends)		input[id*='ser'] (contains)	li:nth-child(2)

Q140. How do you handle dynamic elements?

- Use the stable part of the attribute with contains(), starts-with(), or CSS ^=, \$=, *=.
- Locate through a stable neighbour using axes.
- Use other stable attributes such as name, data-testid, aria-label.
- Add an explicit wait for the element's state.

- Ask developers to add test ids. This is the best long-term fix.

Q141. What is the difference between `findElement` and `findElements`?

	<code>findElement</code>	<code>findElements</code>
Returns	One <code>WebElement</code> (the first match)	<code>List<WebElement></code>
No match	Throws <code>NoSuchElementException</code>	Returns an empty list

To check that an element is absent, use `findElements(...).size() == 0`.

Browser and element commands

Q142. What is the difference between `driver.get()` and `driver.navigate().to()`?

Both open a URL and wait for the page to load. `navigate()` also gives `back()`, `forward()` and `refresh()`, and keeps browser history for those. In practice `get()` is used to open the application.

Q143. What is the difference between `close()` and `quit()`?

`close()` closes the current window only. `quit()` closes all windows and ends the WebDriver session and the driver process. Use `quit()` in teardown.

Q144. What is the difference between `getText()` and `getAttribute("value")`?

`getText()` returns the visible inner text of an element. `getAttribute("value")` returns the text typed into an input field, which `getText()` cannot read. In Selenium 4, `getDomAttribute()` and `getDomProperty()` are the precise replacements for `getAttribute()`.

Q145. What are `isDisplayed()`, `isEnabled()` and `isSelected()`?

`isDisplayed()` checks that the element is visible. `isEnabled()` checks that it can be interacted with. `isSelected()` checks whether a checkbox, radio button or option is selected.

Q146. In how many ways can you click an element or type text?

Click: `element.click()`, `Actions.click()`, `JavaScript arguments[0].click()`, `element.submit()` for forms, and `sendKeys(Keys.ENTER)`. Type: `sendKeys()`, `Actions.sendKeys()`, and `JavaScript arguments[0].value='text'`.

Waits

Q147. What types of waits are there in Selenium?

Wait	Scope	Behaviour
Implicit	Global, set once for the driver	Polls the DOM for up to the given time whenever <code>findElement</code> cannot find an element
Explicit (<code>WebDriverWait</code>)	One element, one condition	Waits until a condition is true or the timeout is reached; polls every 500 ms
Fluent	One element, one condition	Explicit wait with custom polling interval and ignored exceptions

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(By.id("login")));

Wait<WebDriver> fluent = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
```

```
.pollingEvery(Duration.ofSeconds(2))  
.ignoring(NoSuchElementException.class);
```

Q148. What is the difference between implicit and explicit wait? Can you use both?

Implicit wait applies to every `findElement` call and only waits for presence in the DOM. Explicit wait applies to one element and can wait for specific conditions: visible, clickable, text present, invisible, alert present. Mixing them is not recommended because the timeouts can add up unpredictably. In my framework I use explicit waits inside reusable methods.

Q149. Why should you avoid `Thread.sleep()`?

It is a static wait: it always waits the full time even if the element is ready, which slows the suite, and it still fails if the element takes longer. It also throws a checked `InterruptedException`.

Q150. Name common `ExpectedConditions`.

`visibilityOfElementLocated`, `elementToBeClickable`, `presenceOfElementLocated`, `invisibilityOfElementLocated`, `textToBePresentInElement`, `alertIsPresent`, `frameToBeAvailableAndSwitchToIt`, `titleContains`, `urlContains`, `numberOfWindowsToBe`, `stalenessOf`.

Q151. What is page load timeout?

`driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30))` sets the maximum time to wait for a page to load before throwing `TimeoutException`.

Handling web elements

Q152. How do you handle a dropdown?

For a `<select>` tag use the `Select` class:

```
Select s = new Select(driver.findElement(By.id("country")));  
s.selectByVisibleText("India");  
s.selectByValue("IN");  
s.selectByIndex(2);  
List<WebElement> all = s.getOptions();  
s.getFirstSelectedOption();  
s.isMultiple();
```

For a custom dropdown built with `div` or `li`, click to open it, then find the options with `findElements`, loop, and click the one whose text matches. For an auto-suggest box, type the text, wait for the suggestions, and click the matching one.

Q153. How do you handle alerts?

```
Alert alert = driver.switchTo().alert();  
alert.getText();  
alert.accept();           // OK  
alert.dismiss();         // Cancel  
alert.sendKeys("text");  // prompt
```

There are three JavaScript alerts: simple, confirmation and prompt. For a basic authentication pop-up, pass the credentials in the URL: `https://user:pass@site.com`.

Q154. How do you handle frames?

Switch into the frame first, because elements inside an `iframe` are not visible to the driver from the main page.

```
driver.switchTo().frame(0);           // by index  
driver.switchTo().frame("frameName"); // by name or id  
driver.switchTo().frame(frameElement); // by WebElement  
driver.switchTo().parentFrame();       // one level up
```

```
driver.switchTo().defaultContent();           // back to the main page
```

For nested frames, switch to the outer frame and then to the inner one.

Q155. How do you handle multiple windows or tabs?

```
String parent = driver.getWindowHandle();
Set<String> handles = driver.getWindowHandles();
for (String h : handles) {
    if (!h.equals(parent)) {
        driver.switchTo().window(h);
        // work in the child window
        driver.close();
    }
}
driver.switchTo().window(parent);
```

`getWindowHandle()` returns the current window's id as a `String`. `getWindowHandles()` returns a `Set<String>` of all window ids.

Q156. What is the Actions class? What can it do?

It performs advanced mouse and keyboard actions.

```
Actions a = new Actions(driver);
a.moveToElement(menu).perform();           // mouse hover
a.contextClick(el).perform();               // right click
a.doubleClick(el).perform();
a.dragAndDrop(source, target).perform();
a.clickAndHold(src).moveToElement(tgt).release().perform();
a.keyDown(Keys.CONTROL).click(el).keyUp(Keys.CONTROL).perform();
```

`build()` combines several actions into one; `perform()` executes. `perform()` calls `build()` internally.

Q157. What is JavascriptExecutor? When do you use it?

An interface that runs JavaScript in the browser. I use it when normal Selenium commands fail or cannot do the job.

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].click();", el);           // click
js.executeScript("arguments[0].scrollIntoView(true);", el); // scroll to element
js.executeScript("window.scrollTo(0, document.body.scrollHeight);"); // scroll to bottom
js.executeScript("arguments[0].value='abc';", el);       // type
String title = (String) js.executeScript("return document.title;");
js.executeScript("arguments[0].style.border='2px solid red'", el); // highlight
```

Q158. How do you take a screenshot?

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("./screenshots/" + name + ".png"));

File elShot = element.getScreenshotAs(OutputType.FILE); // one element (Selenium 4)
```

In my framework a TestNG listener takes a screenshot in `onTestFailure` and attaches it to the Extent report.

Q159. How do you upload and download a file?

Upload: if the element is `<input type="file">`, send the file path with `sendKeys()`; no click is needed. If it is a native OS dialog, use the Robot class or AutoIT. Download: set the download directory through ChromeOptions preferences, click download, then check with Java File that the file exists.

Q160. How do you handle a web table?

Find the rows and columns with `findElements()`, then loop.

```
List<WebElement> rows = driver.findElements(By.xpath("//table[@id='t1']/tbody/tr"));
for (int i = 1; i <= rows.size(); i++) {
    String name = driver.findElement(By.xpath("//table[@id='t1']/tbody/tr[" + i + "]/td[1]")).getText();
    if (name.equals("Ujjwal")) {
        driver.findElement(By.xpath("//table[@id='t1']/tbody/tr[" + i + "]/td[5]/button")).click();
        break;
    }
}
```

For a paginated table, repeat this inside a loop that clicks "Next" until the record is found or Next is disabled.

Q161. How do you find broken links on a page?

Collect all a tags, read each href, send an HTTP request with `URLConnection`, and check the response code. A code of 400 or above means the link is broken.

```
for (WebElement link : driver.findElements(By.tagName("a"))) {
    String url = link.getAttribute("href");
    if (url == null || url.isEmpty()) continue;
    URLConnection c = (URLConnection) new URL(url).openConnection();
    c.setRequestMethod("HEAD");
    c.connect();
    if (c.getResponseCode() >= 400) System.out.println("Broken: " + url);
}
```

Q162. How do you handle a calendar or date picker?

If the field accepts typing, use `sendKeys()` with the date. Otherwise open the picker, click Next or Previous in a loop until the required month and year are shown, then click the day by its text. As a last option, set the value with JavaScript.

Q163. How do you handle cookies?

```
driver.manage().getCookies(), getCookieNamed("name"), addCookie(new Cookie("k", "v")),
deleteCookieNamed("name"), deleteAllCookies().
```

Q164. What is headless mode? How do you run it?

The browser runs without a visible window. It is faster and is used on CI servers.

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");
options.addArguments("--start-maximized", "--incognito", "--disable-notifications");
options.setAcceptInsecureCerts(true); // SSL certificate errors
WebDriver driver = new ChromeDriver(options);
```

Q165. How do you handle captcha and OTP?

They are designed to stop automation, so we do not automate them through the UI. In test environments we ask developers to disable captcha, provide a fixed test value, or whitelist the test account. For OTP we read it from the database or an API, or use a fixed OTP in the test environment.

Q166. What is Shadow DOM, and how do you handle it?

A shadow DOM is an encapsulated DOM tree inside an element; normal locators cannot see inside it. In Selenium 4, get the shadow root and search inside it with CSS: `element.getShadowRoot().findElement(By.cssSelector("..."))`. XPath does not work inside a shadow root.

Exceptions

Q167. Which Selenium exceptions have you faced, and how did you fix them?

Exception	Cause	Fix
NoSuchElementException	Wrong locator, element not loaded yet, or inside a frame	Correct the locator, add an explicit wait, switch to the frame
StaleElementReferenceException	The element was found, then the DOM refreshed	Find the element again; wait for stalenessOf or re-locate inside a retry
ElementClickInterceptedException	Another element covers it (overlay, loader, sticky header)	Wait for the overlay to disappear, scroll into view, or click with JavaScript
ElementNotInteractableException	Present in the DOM but hidden or disabled	Wait for visibility or clickability
TimeoutException	An explicit wait condition was not met in time	Check the locator and condition, increase the timeout if justified
NoSuchFrameException / NoSuchWindowException	Wrong frame or window reference	Wait for the frame or window, check the handle
NoAlertPresentException	Switching to an alert that is not there	Wait for alertIsPresent
SessionNotCreatedException	Browser and driver versions do not match	Update the driver; use Selenium Manager
InvalidSelectorException	Syntax error in XPath or CSS	Correct the selector
WebDriverException	Browser crashed or driver not reachable	Check the driver and session

Q168. What causes StaleElementReferenceException, in detail?

Selenium stores a reference to an element when it is found. If the page reloads or JavaScript redraws that part of the DOM, the old reference points to an element that no longer exists. It often happens inside loops after an action refreshes a list. The fix is to locate the element again after the change. Page Factory's lazy lookup helps because it finds the element each time it is used.

Page Object Model

Q169. What is the Page Object Model? What are its advantages?

POM is a design pattern in which each page of the application has its own class. The class holds the page's locators and the methods that act on them. Tests call those methods and contain no locators. Advantages: locators live in one place so maintenance is easy, code is reused, tests read clearly, and tests and page logic are separated.

Q170. What is the difference between POM and Page Factory?

POM is the design pattern. Page Factory is a Selenium class that implements it using annotations: `@FindBy` declares elements and `PageFactory.initElements(driver, this)` initialises them. Page Factory uses lazy initialisation: the element is looked up when it is used. With plain POM we declare By locators and call `driver.findElement` ourselves.

```
public class LoginPage {  
    private WebDriver driver;  
  
    @FindBy(id = "username") private WebElement username;
```

```

@FindBy(id = "password") private WebElement password;
@FindBy(xpath = "//button[@type='submit']") private WebElement loginBtn;

public LoginPage(WebDriver driver) {
    this.driver = driver;
    PageFactory.initElements(driver, this);
}

public HomePage login(String user, String pass) {
    username.sendKeys(user);
    password.sendKeys(pass);
    loginBtn.click();
    return new HomePage(driver);
}
}

```

Q171. What are @FindBy, @FindBys, @FindAll and @CacheLookup?

@FindBy locates by one locator. @FindBys applies several locators as AND (parent then child). @FindAll applies several locators as OR. @CacheLookup stores the element after the first lookup; use it only for elements that never change, otherwise it causes stale element errors.

Q172. Many teams now prefer By locators over Page Factory. Why?

By locators combined with explicit waits give more control, avoid stale references from cached elements, and work better for dynamic pages. Page Factory is still fine for stable pages. I can work with both.

Selenium Grid and cross-browser

Q173. What is Selenium Grid?

Grid runs tests on several machines, browsers and operating systems at the same time. A hub receives the test requests and routes them to nodes that have the requested browser. Tests connect with RemoteWebDriver:

```

ChromeOptions options = new ChromeOptions();
WebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), options);

```

Selenium 4 Grid can run in standalone, hub-and-node, or fully distributed mode, and has ready Docker images. Cloud grids such as BrowserStack and Sauce Labs work the same way.

Q174. How do you do cross-browser testing?

I pass the browser name as a TestNG parameter or a Maven property, and a driver factory creates the matching driver. In testng.xml I define one <test> per browser with parallel="tests".

```

public static WebDriver initDriver(String browser) {
    switch (browser.toLowerCase()) {
        case "firefox": return new FirefoxDriver();
        case "edge":    return new EdgeDriver();
        default:        return new ChromeDriver();
    }
}

```

Q175. Can Selenium test APIs, mobile apps or desktop apps?

No. Selenium automates web browsers only. For APIs we use REST Assured or Postman, for mobile Appium, and for desktop tools such as WinAppDriver.

5. TestNG

Q176. What is TestNG? Why use it with Selenium?

TestNG is a Java testing framework. Selenium only drives the browser; TestNG supplies everything around the test: annotations for setup and teardown, assertions, grouping, priorities, dependencies, data-driven tests through DataProvider, parallel execution, listeners, and HTML reports.

Q177. List the TestNG annotations in order of execution.

@BeforeSuite → @BeforeTest → @BeforeClass → @BeforeMethod → @Test → @AfterMethod → @AfterClass → @AfterTest → @AfterSuite.

Annotation	Runs	Typical use
@BeforeSuite / @AfterSuite	Once per suite	Start and flush the report, load config
@BeforeTest / @AfterTest	Once per <test> tag in testng.xml	Browser-level setup for cross-browser runs
@BeforeClass / @AfterClass	Once per class	Class-level data
@BeforeMethod / @AfterMethod	Before and after every @Test	Launch and quit the browser

There are also @BeforeGroups and @AfterGroups.

Q178. What are the attributes of @Test?

priority, enabled, groups, dependsOnMethods, dependsOnGroups, dataProvider, dataProviderClass, invocationCount, threadPoolSize, timeout, expectedExceptions, alwaysRun, description, retryAnalyzer.

Q179. How does priority work?

The lower the number, the earlier the test runs. The default priority is 0. Negative values are allowed. Tests with the same priority run in alphabetical order of method name.

Q180. What is the difference between dependsOnMethods and priority?

priority only sets the order. dependsOnMethods creates a real dependency: if the parent test fails, the dependent test is skipped. Add alwaysRun = true to run it anyway.

Q181. How do you skip or disable a test?

@Test(enabled = false) disables it. To skip at runtime, throw new SkipException("reason").

Q182. What is the difference between hard and soft assertions?

	Hard assert (Assert)	Soft assert (SoftAssert)
On failure	Stops the test immediately	Records the failure and continues
Reporting	At once	Only when assertAll() is called
Use when	The next steps make no sense after a failure	Checking many fields on one page

```
SoftAssert soft = new SoftAssert();
soft.assertEquals(actualTitle, "Home");
soft.assertTrue(logos.isDisplayed());
soft.assertAll();           // without this the test passes even if checks failed
```

Common methods: assertEquals, assertNotEquals, assertTrue, assertFalse, assertNull, assertNotNull.

Q183. What is testng.xml? What can you configure in it?

It is the suite configuration file. The structure is <suite> → <test> → <classes> → <class> → <methods>. In it you choose which classes, packages, methods and groups to run, pass parameters, set parallel execution and thread count, and register listeners.

```
<suite name="Regression" parallel="tests" thread-count="3">
  <listeners>
    <listener class-name="listeners.TestListener"/>
  </listeners>
  <test name="ChromeTests">
    <parameter name="browser" value="chrome"/>
    <groups><run><include name="regression"/></run></groups>
    <classes>
      <class name="tests.LoginTest"/>
    </classes>
  </test>
</suite>
```

Q184. How do you pass parameters to a test?

Two ways. @Parameters reads a value from testng.xml, good for configuration such as browser and environment. @DataProvider supplies many sets of test data from code, good for data-driven tests.

Q185. What is a DataProvider? How do you use it for data-driven testing?

A method annotated with @DataProvider returns Object[][] (or an Iterator<Object[]>). The test runs once per row.

```
@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        {"admin", "admin123", true},
        {"invalid", "wrong", false}
    };
}

@Test(dataProvider = "loginData")
public void loginTest(String user, String pass, boolean expected) {
    // runs twice
}
```

In my framework the DataProvider calls an Excel utility (Apache POI) that reads the sheet into Object[][]. If it lives in another class, use dataProviderClass = TestData.class.

@DataProvider(parallel = true) runs the rows in parallel.

Interview tip: The JD asks for "parameterized and data-driven testing" twice. Be ready to explain DataProvider with Excel, Scenario Outline in Cucumber, and the Postman Collection Runner with a CSV file.

Q186. What are groups? How do you run only smoke tests?

Tag tests with @Test(groups = {"smoke", "regression"}), then include the group in testng.xml or run mvn test -Dgroups=smoke.

Q187. How do you run tests in parallel?

Set parallel and thread-count on the <suite> tag. Values for parallel: methods, classes, tests, instances. For safe parallel runs each thread needs its own driver, so I keep the driver in a ThreadLocal<WebDriver>, and tests must not share data.

Q188. What are listeners? Which have you used?

Listeners are interfaces that react to events during execution. ITestListener has onTestStart, onTestSuccess, onTestFailure, onTestSkipped, onStart, onFinish. I use it to log results to the Extent report and to take a screenshot on failure. Others: ISuiteListener, IRetryAnalyzer, IAnnotationTransformer, IReporter. Register them in testng.xml or with @Listeners.

Q189. How do you re-run failed tests?

Two ways. TestNG writes testng-failed.xml in test-output after a run; running it executes only the failed tests. Or implement IRetryAnalyzer:

```
public class Retry implements IRetryAnalyzer {
    private int count = 0;
    private static final int MAX = 2;
    public boolean retry(ITestResult result) {
        if (count < MAX) { count++; return true; }
        return false;
    }
}
// @Test(retryAnalyzer = Retry.class)
```

To apply it to every test without editing each one, use IAnnotationTransformer. Retry is for genuine flakiness only; a real failure must be investigated, not hidden.

Q190. What do invocationCount, timeout and expectedExceptions do?

invocationCount = 5 runs the test five times. timeout = 3000 fails the test if it takes more than 3 seconds. expectedExceptions = ArithmeticException.class passes the test only if that exception is thrown.

Q191. What is the difference between TestNG and JUnit?

TestNG has richer annotations (suite, test, groups), built-in grouping, dependencies, DataProvider, parallel execution through XML, and built-in reports. JUnit 5 has closed much of the gap, but TestNG remains more common in Selenium frameworks.

6. Cucumber and BDD

Q192. What is BDD? What is Cucumber?

Behaviour Driven Development describes the behaviour of the application in plain language that business people, developers and testers all understand. Cucumber is a tool that supports BDD: it reads scenarios written in Gherkin and runs the code linked to each step.

Q193. What are the advantages of BDD?

It gives one shared language, requirements become executable tests, scenarios act as living documentation, steps are reusable, and non-technical people can read and review tests.

Q194. What is Gherkin? What are its keywords?

Gherkin is the plain-text language for writing scenarios. Keywords: Feature, Scenario, Given (precondition), When (action), Then (expected result), And, But, Background, Scenario Outline, Examples, and *.

```
Feature: Login

  Background:
    Given the user is on the login page

  @smoke
  Scenario: Successful login
    When the user logs in with "admin" and "admin123"
    Then the home page is displayed

  @regression
  Scenario Outline: Login with invalid data
    When the user logs in with "<username>" and "<password>"
    Then the error "<message>" is shown

  Examples:
    | username | password | message           |
    | admin    | wrong    | Invalid credentials |
    |          | admin123 | Username required  |
```

Q195. What are the main files in a Cucumber framework?

- Feature file (.feature): scenarios in Gherkin.
- Step definition class: Java methods mapped to steps with @Given, @When, @Then.
- Test runner class: configures and starts execution with @CucumberOptions.
- Hooks class: @Before and @After setup and teardown.

Q196. What is a step definition? How does Cucumber match a step to it?

A step definition is a Java method annotated with a Cucumber expression or regular expression that matches the step text. Values in the step are passed as parameters.

```
@When("the user logs in with {string} and {string}")
public void login(String user, String pass) {
    loginPage.login(user, pass);
}
```

Cucumber expression types: {string}, {int}, {float}, {word}.

Q197. What is the difference between Scenario and Scenario Outline?

A Scenario runs once with fixed values. A Scenario Outline runs once for each row of its Examples table, with placeholders in angle brackets. It is Cucumber's way of data-driven testing.

Q198. What is Background?

Steps that are common to every scenario in a feature file, written once. They run before each scenario, after the @Before hooks.

Q199. What is a DataTable? How is it different from Examples?

A DataTable passes a table of data to one step, and the scenario runs once. Examples makes the whole scenario run once per row.

```
When the user fills the form
| name | Ujjwal |
| city | Delhi |

@When("the user fills the form")
public void fill(DataTable table) {
    Map<String, String> data = table.asMap(String.class, String.class);
    // or table.asMaps() for a table with a header row
}
```

Q200. Explain the test runner and @CucumberOptions.

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue = {"stepdefinitions", "hooks"},
    tags = "@smoke and not @wip",
    plugin = {"pretty", "html:target/cucumber.html", "json:target/cucumber.json",
            "rerun:target/failed.txt"},
    dryRun = false,
    monochrome = true
)
public class TestRunner extends AbstractTestNGCucumberTests { }
```

Option	Meaning
features	Path to the feature files
glue	Packages of step definitions and hooks
tags	Which scenarios to run
plugin	Report formats
dryRun	true checks that every step has a definition without running the tests
monochrome	Readable console output

Q201. What are hooks?

Blocks of code that run before and after each scenario or step: @Before, @After, @BeforeStep, @AfterStep (and @BeforeAll, @AfterAll). I launch the browser in @Before and, in @After, take a screenshot if the scenario failed and quit the browser. Hooks can be ordered with order and limited to tags: @Before("@ui").

```
@After
public void tearDown(Scenario scenario) {
    if (scenario.isFailed()) {
        byte[] shot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
        scenario.attach(shot, "image/png", scenario.getName());
    }
    driver.quit();
}
```

Q202. What are tags? How do you run specific scenarios?

Tags are labels such as @smoke on a feature or scenario. Run with tag expressions: @smoke and @login, @smoke or @regression, not @wip. From Maven: mvn test -Dcucumber.filter.tags="@smoke".

Q203. How do you share data between step definition classes?

With dependency injection, using cucumber-picocontainer. I create a context class that holds the driver and page objects and pass it through the constructor of each step definition class. Cucumber creates one instance per scenario, so state is shared within a scenario and isolated between scenarios.

Q204. How do you run Cucumber scenarios in parallel, and re-run failed ones?

Parallel: with TestNG, override the `scenarios()DataProvider` in the runner with `@DataProvider(parallel = true)`. Re-run: the rerun plugin writes failed scenarios to a text file; a second runner points `features = "@target/failed.txt"`.

Q205. Which reports does Cucumber produce?

Built-in HTML, JSON and JUnit XML reports through plugins. For richer reports I use the Extent Cucumber adapter or Allure.

Q206. What is the difference between TDD and BDD?

In TDD developers write a failing unit test first, then the code to pass it; it is code-focused. BDD extends this to behaviour written in plain language from the user's point of view, involving business, developers and testers.

Q207. What are the drawbacks of Cucumber?

It adds a layer to maintain (feature files and step definitions), step wording must stay consistent or you get duplicate and ambiguous steps, and it brings little value if business people never read the scenarios.

7. Automation Framework Design

Q208. Explain your automation framework.

Interview tip: This is the most important answer of the interview. Speak for two to three minutes in this order: type, tech stack, structure, flow, reporting, CI, and one improvement you made. Adjust the details to what you really built.

"I work on a hybrid framework built with Selenium WebDriver and Java. It combines the Page Object Model with data-driven testing, and uses TestNG as the test runner and Maven as the build tool.

The structure is layered. There is a base layer with a driver factory that creates the browser from the config and stores it in a ThreadLocal for parallel runs. The page layer has one class per page with locators and business methods. The test layer has TestNG classes that call page methods and hold assertions. The utilities layer has reusable code: wait helpers, an Excel reader using Apache POI, a config reader for properties files, a screenshot utility, and JavaScript helpers. Test data is kept outside the code in Excel and properties files.

For reporting I use Extent Reports with a TestNG listener that logs each step and attaches a screenshot on failure, and Log4j for logs. There is a retry analyzer for flaky tests. We also have a Cucumber layer for BDD scenarios, with feature files, step definitions and hooks reusing the same page classes.

The code is in Git. Jenkins triggers the suite with a Maven command, with parameters for browser, environment and suite, and publishes the report and emails it. Smoke runs on every build and full regression runs nightly.

My contributions were [for example: adding parallel execution which cut the run time from X to Y, building the Excel data utility, and stabilising flaky tests by replacing sleeps with explicit waits]."

Q209. Draw or describe your framework's folder structure.

```
project
├── src/main/java
│   ├── base           BaseTest, DriverFactory
│   ├── pages          LoginPage, HomePage ...
│   ├── utils          WaitUtil, ExcelUtil, ConfigReader, ScreenshotUtil
│   ├── listeners      TestListener, RetryAnalyzer
│   └── constants      FrameworkConstants
├── src/test/java
│   ├── tests          LoginTest, OrderTest ...
│   ├── stepdefinitions, hooks, runners    (Cucumber)
├── src/test/resources
│   ├── features       *.feature
│   ├── testdata       TestData.xlsx
│   ├── config         config.properties, qa.properties
│   └── log4j2.xml
├── testng.xml, smoke.xml, regression.xml
├── pom.xml
└── reports, screenshots, logs
```

Q210. What types of automation frameworks are there?

- Linear (record and playback): no reuse.
- Modular: the application is split into modules with separate scripts.
- Data-driven: test data is kept outside the script; the same test runs with many data sets.
- Keyword-driven: actions are keywords stored in a table.
- Hybrid: a combination, usually POM + data-driven + utilities. This is the most common.
- BDD: scenarios in Gherkin with Cucumber.

Q211. Which design patterns have you used?

- Page Object Model: one class per page.
- Singleton: one config reader, one driver instance per thread.
- Factory: DriverFactory returns the right driver for the browser name.
- Builder or fluent pattern: page methods return the next page object so calls can chain.

Q212. How do you read data from Excel?

With Apache POI. XSSFWorkbook for .xlsx and HSSFWorkbook for .xls. The order is Workbook → Sheet → Row → Cell.

```
FileInputStream fis = new FileInputStream("./testdata/TestData.xlsx");
Workbook wb = new XSSFWorkbook(fis);
Sheet sheet = wb.getSheet("Login");
int rows = sheet.getLastRowNum();
int cols = sheet.getRow(0).getLastCellNum();
Object[][] data = new Object[rows][cols];
DataFormatter fmt = new DataFormatter();
for (int i = 1; i <= rows; i++) {
    for (int j = 0; j < cols; j++) {
        data[i - 1][j] = fmt.formatCellValue(sheet.getRow(i).getCell(j));
    }
}
wb.close();
```

Q213. How do you read a properties file?

```
Properties prop = new Properties();
FileInputStream fis = new FileInputStream("./src/test/resources/config/config.properties");
prop.load(fis);
String url = prop.getProperty("url");
```

Q214. How do you run the same tests on different environments?

One properties file per environment (qa.properties, stage.properties). The environment name comes from a Maven property: mvn test -Denv=qa, read with System.getProperty("env"), and the config reader loads the matching file. In Jenkins it is a build parameter.

Q215. How do you handle sensitive data such as passwords?

Never hard-code or commit them. I keep them in environment variables or Jenkins credentials, and pass them at runtime. Config files in Git hold only non-sensitive values.

Q216. How do you generate reports and take screenshots on failure?

Extent Reports. In the listener's onStart I create the report, in onTestStart a test entry, in onTestFailure I log the exception, take a screenshot with TakesScreenshot and attach it, and in onFinish I call flush(). For parallel runs the ExtentTest object is kept in a ThreadLocal. Allure is an alternative.

Q217. How do you make WebDriver thread-safe for parallel execution?

```
public class DriverFactory {
    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static void setDriver(WebDriver d) { driver.set(d); }
    public static WebDriver getDriver() { return driver.get(); }
    public static void unload() { driver.remove(); }
}
```

Each thread gets its own browser. Tests must also be independent: no shared static data and no reliance on execution order.

Q218. What makes tests flaky, and how do you fix them?

Causes: timing and synchronisation problems, dynamic locators, test data shared between tests, dependency on test order, environment instability, and animations or overlays. Fixes: explicit waits in place of sleeps, stable locators, independent tests that create their own data, isolating state between tests, and retry only as a safety net. I track flaky tests and fix the root cause.

Q219. Which test cases do you automate, and which do you not?

Automate: regression and smoke tests, repetitive and data-heavy tests, stable features, and critical business flows. Do not automate: features that change often, one-time tests, usability and look-and-feel checks, exploratory testing, and captcha or OTP flows.

Q220. What is the test automation pyramid?

Many fast unit tests at the base, fewer API or integration tests in the middle, and the fewest UI tests at the top, because UI tests are the slowest and most fragile. A good strategy pushes checks down to the API layer wherever possible.

Q221. How do you measure the value of automation?

Automation coverage, regression execution time before and after, defects caught by automation, flaky test rate, pass rate trend, and maintenance effort per sprint.

Q222. How do you keep a regression suite maintainable as it grows?

- Follow POM strictly and keep locators in one place.
- Review automation code like product code, with pull requests.
- Tag tests (smoke, regression, module) so subsets can run.
- Remove or fix obsolete and flaky tests each sprint.
- Run in parallel and in headless mode to keep feedback fast.
- Update scripts in the same sprint as the feature change.

Q223. A test passes locally but fails in Jenkins. How do you debug it?

I compare the two environments: browser and driver version, headless mode and screen resolution, test data, environment URL, and timing, since CI machines are slower. I read the Jenkins console log, the screenshot and the report. Typical fixes are setting a window size in headless mode, adding proper waits, and removing dependence on local files or data.

Q224. 30 of 200 tests failed in last night's run. What do you do?

I group failures by error message first. If many share one error, there is one cause: the environment was down, login failed, or a common locator changed. I check the screenshots and logs, re-run the failures, and separate them into script issues, environment issues, and real defects. I log the real defects, fix the scripts, and report the summary to the team.

8. API Testing and Postman

Interview tip: Sections marked "practised" are things you have done hands-on in Postman this week, so answer them with confidence and your own examples. The part titled "Also asked by this JD" covers topics you have not practised yet. Know the concept and the code shape, and say honestly that your hands-on depth there is still growing.

API fundamentals (practised)

Q225. What is an API? What is API testing?

An API (Application Programming Interface) lets two software systems communicate. API testing sends requests directly to the API and validates the response, without the UI. It tests the business logic layer.

Q226. Why is API testing important? How is it different from UI testing?

It can start before the UI exists, runs in milliseconds, is stable because it does not depend on screens, finds defects at the source, and can send inputs the UI would block. UI testing checks the look and the user flow; API testing checks logic and data.

Q227. What is REST? What are its principles?

REST is an architectural style in which every resource has a URL and is acted on with standard HTTP methods. Principles: client-server separation, statelessness (each request carries everything needed), uniform interface, resource-based URLs, cacheable responses, and a layered system.

Q228. What is the difference between REST and SOAP?

	REST	SOAP
Type	Architectural style	Protocol
Format	JSON, XML, text	XML only
Weight	Light and fast	Heavy
Contract	OpenAPI (Swagger), optional	WSDL, mandatory
Use	Web and mobile apps	Older banking and enterprise systems

Q229. What are the parts of an HTTP request and response?

Request: URL, method, headers, body. Response: status code, headers, body.

Q230. Explain the HTTP methods.

Method	Action	Body	Idempotent	Typical success code
GET	Read	No	Yes	200
POST	Create	Yes	No	201
PUT	Replace the whole resource	Yes	Yes	200
PATCH	Update part of a resource	Yes	Not guaranteed	200
DELETE	Remove	Usually no	Yes	200 or 204

Others: HEAD returns headers only; OPTIONS returns the methods a URL allows.

Q231. What is the difference between PUT and PATCH? POST and PUT?

PUT replaces the entire resource, so you send the complete record; PATCH changes only the fields you send. POST creates a new resource on a collection URL and the server assigns the id; PUT replaces an existing resource at a specific URL. POST is not idempotent; PUT is.

Q232. What is idempotency?

A method is idempotent if sending the same request many times leaves the server in the same state as sending it once. GET, PUT and DELETE are idempotent; POST is not. It matters for retries: a retried payment POST must not charge twice.

Q233. Explain the status code classes and the important codes.

Code	Meaning
200 OK	Success with a body
201 Created	A resource was created
204 No Content	Success, no body
301 / 304	Moved permanently / Not modified
400 Bad Request	Malformed request: invalid JSON, missing field
401 Unauthorized	Not authenticated: token missing or invalid
403 Forbidden	Authenticated but not permitted
404 Not Found	Resource or URL does not exist
405 Method Not Allowed	URL exists, method not supported
409 Conflict	Clashes with existing data, such as a duplicate
415 Unsupported Media Type	Body format not supported
422 Unprocessable Entity	Valid JSON that breaks a business rule
429 Too Many Requests	Rate limit hit
500 / 502 / 503 / 504	Server error / bad gateway / unavailable / gateway timeout

1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

Q234. What is the difference between 401 and 403? Between 400 and 404?

401 means the server does not know who you are. 403 means it knows and still refuses. 400 means the request itself is malformed. 404 means the request is fine but the resource is not there.

Q235. A search returns no results. Should the API return 404?

No. A filter or search with no match returns 200 with an empty array, because the request succeeded. 404 is for a specific resource requested by id that does not exist.

Q236. Does POST always return 201?

No. The status code reflects what the server did. A POST that creates a record returns 201; a POST that performs an action such as login or search returns 200. The API documentation decides.

Q237. What is the difference between a path parameter and a query parameter?

	Path parameter	Query parameter
Job	Identifies one resource	Filters, sorts, paginates, searches
Example	/posts/1	/posts?userId=1&_limit=5
Required	Yes	Usually optional
Returns	One object	An array
Nothing found	404	200 with []

Q238. What are headers? Name the important ones.

Key-value pairs that describe the message. Content-Type is the format of the body being sent. Accept is the format the client wants back. Authorization carries credentials or a token. User-Agent, Host, Cache-Control, Cookie and Set-Cookie are also common. Header names are not case-sensitive. A wrong Content-Type should give 415 or 400.

Q239. What is JSON? What data types does it have?

A lightweight text format of key-value pairs. Types: string, number, boolean, null, object {}, array []. Keys are in double quotes and there are no trailing commas.

Q240. What is URL encoding?

Replacing spaces and special characters in a URL with percent codes, such as %20 for a space, because characters like &, = and ? have a structural meaning in a URL.

Q241. Why should sensitive data not be sent in query parameters?

URLs are stored in browser history, server logs and proxies. Sensitive data belongs in headers or the body, over HTTPS.

Postman (practised)

Q242. What is Postman? What have you done with it?

Postman is an API client for building, sending, saving and automating API requests. I have built collections with folders for CRUD flows, used query and path parameters, headers and all body types, created collection, environment and global variables, written test scripts for status, time, headers, body and arrays, added negative tests, and stored the collection in Git.

Q243. What body types does Postman support?

none, form-data (multipart, can carry files), x-www-form-urlencoded (text key-value pairs in one encoded line), raw (JSON, text, XML), binary (one file), and GraphQL.

Q244. What is the difference between form-data and x-www-form-urlencoded?

Both send key-value pairs. Urlencoded sends one encoded line and supports only text. Form-data sends each field as a separate part divided by a boundary, so it supports file upload.

Q245. What is a collection? What is an environment?

A collection is a group of saved requests, organised in folders, that can be shared and run together. An environment is a named set of variables, such as baseUrl and credentials, that lets the same collection run against dev, QA or production by switching one dropdown.

Q246. What variable scopes does Postman have, and what is the precedence?

Global, collection, environment, data, and local. The narrowest scope wins: local, then data, then environment, then collection, then global. Variables are used as `{{name}}`.

Q247. What is the difference between the initial (shared) and current (local) value?

The initial value is synced and visible to the team. The current value stays on my machine and is the one Postman actually sends. Secrets go only in the current value.

Q248. What are dynamic variables?

Built-in variables that generate fresh data on every send, such as `{{$randomInt}}`, `{{$guid}}`, `{{$timestamp}}`, `{{$randomEmail}}`. They help create unique test data.

Q249. How do you write tests in Postman?

In the Scripts tab, in the post-response (After response) section, using JavaScript and the `pm` object.

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response time is below 1000 ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(1000);
});

pm.test("Content-Type is JSON", function () {
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

const data = pm.response.json();

pm.test("Body values and types are correct", function () {
  pm.expect(data.id).to.equal(1);
  pm.expect(data).to.have.property("title");
  pm.expect(data.title).to.be.a("string");
});

// On a request that returns a list, such as GET /posts?userId=1
const posts = pm.response.json();
pm.test("Every post belongs to the user", function () {
  pm.expect(posts).to.be.an("array").with.lengthOf(10);
  posts.forEach(function (post) {
    pm.expect(post.userId).to.equal(Number(pm.environment.get("userId")));
  });
});
```

Q250. What is the difference between `equal` and `eql`?

`equal` is a strict comparison for single values. `eql` is a deep comparison that checks whether two objects or arrays have the same content.

Q251. How do you avoid repeating the same test in every request?

Write it once in the collection's or folder's Scripts tab; it runs for every request inside. The order is collection, folder, then request.

Q252. How do you debug a failing request or test?

Read the assertion message, print values with `console.log`, and inspect the exact request and response in the Postman Console.

Q253. What do you validate in an API response?

Status code, response body values, data types and structure (schema), headers, response time, and for failures the error message. For write operations I also verify with a follow-up GET or a database check.

Q254. How do you test a CRUD flow end to end?

Create with POST and capture the id, read it with GET, update with PUT or PATCH, read again to confirm the change, delete, then read again and expect 404. Each GET verifies the step before it.

Q255. What negative tests do you write for an API?

- Missing required fields, wrong data types, empty body, invalid JSON.
- Boundary values: very long strings, zero, negative numbers, special characters.
- Non-existent id (404), wrong method (405), wrong Content-Type (415).
- No token (401), expired token (401), another user's token or a lower role (403).
- Duplicate creation (409).
- Injection strings in fields.

A good API returns the right 4xx code with a clear message, and never a 500.

Also asked by this JD

Q256. What is a pre-request script?

JavaScript that runs before the request is sent. It is used to prepare data: generate a timestamp or random value, compute a signature, or set a variable.

```
pm.environment.set("email", "user" + Date.now() + "@test.com");
```

Q257. What is request chaining? How do you do it?

Passing data from one response to the next request. In the first request's test script I save a value into a variable, and the next request uses it.

```
// Login request, post-response script
const res = pm.response.json();
pm.environment.set("token", res.token);
pm.environment.set("userId", res.id);

// Next request: URL {{baseUrl}}/users/{{userId}}, header Authorization: Bearer {{token}}
```

Q258. What is the difference between authentication and authorization?

Authentication verifies who you are. Authorization verifies what you are allowed to do.

Q259. What authentication types do you know?

Type	How it works
API key	A key sent in a header or query parameter
Basic auth	Username and password, Base64 encoded, in the Authorization header
Bearer token	A token sent as Authorization: Bearer <token>
JWT	A signed token with three parts: header, payload, signature. It carries claims and an expiry.
OAuth 2.0	The client gets an access token from an authorization server and uses it to call the API. Common grant types: authorization code, client credentials. Refresh tokens renew access tokens.

In Postman, auth is set in the Authorization tab, ideally once at collection level so every request inherits it.

Q260. What is JSON schema validation? How do you do it in Postman?

A JSON schema describes the structure of a response: field names, data types, required fields. Schema validation checks the contract, not the values, so it catches a missing field or a changed type.

```
const schema = {
  type: "object",
  required: ["id", "title", "userId"],
  properties: {
    id: { type: "number" },
    title: { type: "string" },
    userId: { type: "number" }
  }
};
pm.test("Schema is valid", function () {
  pm.response.to.have.jsonSchema(schema);
});
```

Q261. How do you do data-driven testing in Postman?

With the Collection Runner and a CSV or JSON data file. Each row is one iteration, and the columns are available as {{columnName}} in requests and as pm.iterationData.get("columnName") in scripts.

Q262. What is the Collection Runner?

It runs all requests of a collection or folder in order, for a chosen environment, number of iterations and data file, and shows pass and fail results. pm.execution.setNextRequest("name") (older: postman.setNextRequest) changes the order during a run.

Q263. What is Newman? How do you run a collection in CI?

Newman is Postman's command-line runner. It runs an exported collection from a terminal or a pipeline.

```
npm install -g newman newman-reporter-htmlextra
newman run "User Management API.postman_collection.json" \
  -e QA.postman_environment.json \
  -d data.csv \
  -r cli,htmlextra,junit
```

In Jenkins this command is a build step. Newman exits with a non-zero code when a test fails, so the build fails, and the HTML and JUnit reports are published.

Q264. What are mock servers and monitors in Postman?

A mock server returns saved example responses, so front-end or test work can start before the real API is ready. A monitor runs a collection on a schedule from Postman's cloud and alerts on failures.

Q265. How would you test pagination and rate limiting?

Pagination: first page, last page, a page beyond the end, limit of zero, negative and very large limits, and no record repeated or skipped between pages. Rate limiting: send requests above the limit quickly, expect 429 with a Retry-After header, and confirm access returns after the window.

Q266. What is the difference between API testing with Postman and with REST Assured?

Postman is a GUI tool with JavaScript tests, quick to start and good for exploration and collection-based automation with Newman. REST Assured is a Java library, so API tests live in the same code base, language and CI job as the Selenium framework, with full programming power.

Q267. Show a basic REST Assured test.

```
given()
    .baseUrl("https://jsonplaceholder.typicode.com")
    .header("Content-Type", "application/json")
    .body("{\"title\":\"post\",\"body\":\"text\",\"userId\":1}")
.when()
    .post("/posts")
.then()
    .statusCode(201)
    .body("userId", equalTo(1))
    .body("id", notNullValue())
    .time(lessThan(2000L));

// extract a value for chaining
int id = given().when().get("/posts/1").then().extract().path("id");
```

given() sets up the request, when() sends it, then() validates. Schema validation uses `matchesJsonSchemaInClasspath("schema.json")`. Request and response bodies can be mapped to POJOs with Jackson (serialization and deserialization).

Q268. What is Swagger or OpenAPI?

A standard specification that documents an API: endpoints, methods, parameters, request and response schemas, and auth. Testers use it as the source of truth for expected behaviour, and it can be imported into Postman to generate a collection.

Q269. What is the difference between API testing and contract testing?

API testing checks that the API behaves correctly. Contract testing checks that the provider and its consumers agree on the request and response format, so a provider change does not silently break a consumer. Pact is a common tool.

Q270. What is a microservice? How does it change testing?

An application built as small, independent services that talk over APIs. Testing moves towards the API layer: each service is tested on its own with mocks for its dependencies, then contract and integration tests cover the links between services.

9. Playwright (Basics)

Q271. What is Playwright?

Playwright is an open-source automation framework from Microsoft for web testing. It supports Chromium, Firefox and WebKit, and languages TypeScript, JavaScript, Python, Java and C#. It has its own test runner, Playwright Test, with assertions, parallel execution, reports and tracing built in.

Q272. What is the difference between Selenium and Playwright?

	Selenium	Playwright
Communication	W3C WebDriver protocol over HTTP, through a browser driver	Direct connection to the browser over a WebSocket
Waiting	Manual: implicit, explicit, fluent	Auto-waiting built in
Browsers	Chrome, Firefox, Edge, Safari	Chromium, Firefox, WebKit
Test runner	Needs TestNG or JUnit	Built in
Parallel runs	Through TestNG or Grid	Built in, with workers
Network mocking, API testing	Limited	Built in
Debugging	Screenshots, logs	Trace viewer, video, screenshots, codegen
Maturity	Very large community, long history	Newer, growing fast

Q273. Explain Playwright's architecture.

The test talks to a Playwright server over a single WebSocket connection, and the server drives the browser through its debugging protocol (CDP for Chromium). There is no separate driver per command and no HTTP request per action, which makes it faster and less flaky.

Q274. What are Browser, BrowserContext and Page?

Browser is one browser instance. BrowserContext is an isolated session inside it, like an incognito profile, with its own cookies and storage. Page is one tab in a context. Each test gets a fresh context, which gives isolation without the cost of launching a new browser.

Q275. What is auto-waiting?

Before acting on an element, Playwright automatically waits until it is attached, visible, stable, enabled and able to receive events. This removes most explicit waits. Assertions such as `expect(locator).toBeVisible()` also retry until the timeout.

Q276. Which locators does Playwright recommend?

User-facing locators: `getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, `getByAltText`, `getByTitle`, `getByTestId`. CSS and XPath work through `page.locator()`. Locators are lazy and strict: they are resolved when used, and an action fails if the locator matches more than one element.

Q277. Write a basic Playwright test.

```
import { test, expect } from '@playwright/test';

test('user can log in', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.getByLabel('Username').fill('admin');
  await page.getByLabel('Password').fill('admin123');
  await page.getByRole('button', { name: 'Login' }).click();
  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByText('Welcome')).toBeVisible();
});
```

Q278. What are fixtures and hooks?

Fixtures are objects Playwright sets up and passes to each test, such as page, context, browser and request. You can write custom fixtures, for example a logged-in page. Hooks are test.beforeEach, test.afterEach, test.beforeAll, test.afterAll. Tests are grouped with test.describe.

Q279. What goes in playwright.config.ts?

testDir, timeout, retries, workers, reporter, use (base URL, headless, screenshot, video, trace), and projects to run the same tests on Chromium, Firefox and WebKit.

Q280. How do you handle frames, new tabs, alerts and file upload?

```
await page.frameLocator('#frame').getByText('Submit').click(); // frame

const [newPage] = await Promise.all([ // new tab
  context.waitForEvent('page'),
  page.getByText('Open').click()
]);

page.on('dialog', dialog => dialog.accept()); // alert

await page.setInputFiles('input[type="file"]', 'data/file.pdf'); // upload
```

Q281. How do you run tests and debug failures?

npx playwright test, with options --headed, --project=chromium, --grep @smoke, --workers=4, --debug, and --ui. npx playwright codegen <url> records a script. npx playwright show-report opens the HTML report. The trace viewer shows each action with a DOM snapshot, network calls and console logs.

Q282. How does Playwright run tests in parallel?

Test files run in parallel across worker processes by default. fullyParallel: true also runs tests within a file in parallel. Each test has its own context, so they are isolated.

Q283. Can Playwright test APIs and mock network calls?

Yes. The request fixture sends API calls: const res = await request.get('/posts/1'); expect(res.status()).toBe(200);. page.route() intercepts a request and can mock, modify or block it.

Q284. How do you reuse a login across tests?

Log in once in a setup step, save the session with context.storageState({ path: 'auth.json' }), and load it in the config with use: { storageState: 'auth.json' }.

Q285. Why would you choose Playwright over Selenium, or the reverse?

Playwright for a new project on a modern web app that needs speed, auto-waiting, API and network control, and built-in tooling. Selenium where a team already has a mature Java framework, needs real Safari or older browser coverage, or relies on its wide ecosystem. Many teams run both.

10. Git, Maven, Jenkins, CI/CD and Docker

Git

Q286. What is Git? How is it different from GitHub?

Git is a distributed version control system that tracks changes to code on your machine. GitHub, GitLab and Bitbucket are hosting services for Git repositories with collaboration features such as pull requests.

Q287. Which Git commands do you use daily?

Command	Purpose
<code>git clone <url></code>	Copy a remote repository
<code>git status</code>	Show changed files
<code>git add .</code>	Stage changes
<code>git commit -m "msg"</code>	Save staged changes locally
<code>git push / git pull</code>	Send to / fetch and merge from remote
<code>git branch, git checkout -b name</code>	List / create and switch branch
<code>git merge branch</code>	Merge a branch into the current one
<code>git log --oneline</code>	History
<code>git diff</code>	Show changes
<code>git stash / git stash pop</code>	Set changes aside / bring them back

Q288. What is the difference between `git pull` and `git fetch`?

`fetch` downloads the remote changes without touching your working code. `pull` is `fetch` followed by `merge`.

Q289. What is the difference between merge and rebase?

`Merge` joins two branches and creates a merge commit, keeping the full history. `Rebase` moves your commits on top of the target branch, giving a straight history, but it rewrites commits, so it should not be used on shared branches.

Q290. What is a merge conflict? How do you resolve one?

It happens when two branches change the same lines. Git marks the file with conflict markers. I open the file, keep the correct code, remove the markers, then `git add` and `git commit`.

Q291. What is the difference between `git reset` and `git revert`?

`reset` moves the branch pointer back and can discard commits; use it on local commits. `revert` creates a new commit that undoes an earlier one; it is safe for shared branches.

Q292. What is `.gitignore`? What is a pull request? What is `git cherry-pick`?

`.gitignore` lists files Git should not track, such as `target/`, reports, and IDE files. A pull request asks to merge a branch and is where code review happens. `cherry-pick` applies one specific commit from another branch.

Q293. What branching strategy does your team follow?

A common one: main for production code, develop for integration, a feature/ branch per story, merged through a pull request after review and a green pipeline. Automation code follows the same flow.

Maven

Q294. What is Maven? Why use it?

A build and dependency management tool. It downloads the libraries declared in pom.xml, gives a standard project structure, and compiles, tests and packages the project with one command, which is what CI tools call.

Q295. What is pom.xml?

The Project Object Model file. It holds the project coordinates (groupId, artifactId, version), dependencies, plugins, properties and profiles.

Q296. What is the Maven build lifecycle?

validate → compile → test → package → verify → install → deploy. Running a phase runs all phases before it. mvn clean deletes the target folder.

Q297. Which Maven commands do you use?

```
mvn clean test
mvn clean test -DsuiteXmlFile=regression.xml
mvn test -Dbrowser=chrome -Denv=qa
mvn test -Dgroups=smoke
mvn test -Dcucumber.filter.tags="@smoke"
mvn clean install -DskipTests
```

Q298. What does the Surefire plugin do?

It runs the tests during the test phase and produces reports. In its configuration I point it to testng.xml through suiteXmlFiles. The Failsafe plugin is its counterpart for integration tests.

Q299. What are the Maven repositories and dependency scopes?

Repositories: local (.m2 on your machine), central (public), and remote (company Nexus or Artifactory). Scopes: compile (default), test (only for tests), provided (supplied at runtime by the container), runtime.

Jenkins and CI/CD

Q300. What are CI and CD?

Continuous Integration: developers merge code often, and each merge triggers an automatic build and tests, so problems are found early. Continuous Delivery: every successful build is ready to release, with a manual approval to deploy. Continuous Deployment: every successful build is deployed to production automatically.

Q301. What is Jenkins?

An open-source automation server used to build CI/CD pipelines. It pulls code from Git, builds it, runs tests, publishes reports and sends notifications.

Q302. How did you integrate your automation suite with Jenkins?

- Create a job (freestyle or pipeline) and connect the Git repository with credentials.
- Add parameters for browser, environment and suite.

- Add the build step: `mvn clean test -Dbrowser=$BROWSER -Denv=$ENV`.
- Set a trigger: after the application build, on a schedule (nightly), or on a push.
- Post-build: publish the HTML or Extent report, archive screenshots, and send an email.

Tests run in headless mode on the Jenkins agent.

Q303. What build triggers does Jenkins have?

Manual, build periodically (cron syntax such as `H 2 * * *` for about 2 a.m. daily), poll SCM, webhook from GitHub or Bitbucket on push, and trigger after another job.

Q304. What is a Jenkins pipeline? What is a Jenkinsfile?

A pipeline defines the whole build process as code. The Jenkinsfile is that code, stored in the repository. Declarative syntax is the common style.

```
pipeline {
  agent any
  parameters { choice(name: 'ENV', choices: ['qa', 'stage'], description: 'Environment') }
  stages {
    stage('Checkout') { steps { git branch: 'main', url: 'https://github.com/org/repo.git' } }
    stage('Build')     { steps { sh 'mvn clean compile' } }
    stage('Test')      { steps { sh "mvn test -Denv=${params.ENV}" } }
  }
  post {
    always { junit 'target/surefire-reports/*.xml' }
    failure { echo 'Tests failed' }
  }
}
```

Q305. What is the difference between a freestyle job and a pipeline job?

A freestyle job is configured through the UI and suits simple tasks. A pipeline job is written as code in a Jenkinsfile, is version controlled, and supports stages, parallel steps and complex flows.

Q306. What are Jenkins agents (master and slave)?

The controller schedules jobs and the agents execute them. Agents let builds run on different machines and operating systems, and in parallel.

Q307. Where does testing fit in a CI/CD pipeline?

Unit tests run on every commit. After deployment to a test environment, API tests and smoke UI tests run as a quality gate. Full regression runs nightly or before release. Performance and security scans run on a schedule or before release. A failed gate stops the pipeline.

Docker and cloud (basics)

Q308. What is Docker? Why is it useful for testing?

Docker packages an application with its dependencies into a container that runs the same way everywhere. For testing it gives identical, disposable environments: Selenium Grid in containers, the application under test and its database started on demand, and the test suite itself running in a container in CI.

Q309. What is the difference between an image and a container? Between a container and a virtual machine?

An image is a read-only template; a container is a running instance of an image. A virtual machine includes a full operating system and is heavy; a container shares the host's kernel and is light and fast to start.

Q310. Which Docker commands do you know?

`docker pull`, `docker build -t name .`, `docker run -d -p 4444:4444 selenium/standalone-chrome`, `docker ps`, `docker stop`, `docker images`, `docker logs`, and `docker-compose up` to start several containers together. A Dockerfile is the recipe for an image.

Q311. Have you run tests on the cloud?

Answer honestly. If yes, describe it. If not: "I have run suites on Jenkins. I understand cloud grids such as BrowserStack connect through RemoteWebDriver with capabilities for browser and OS, and that CI agents often run on AWS or Azure. I have not set one up myself."

11. Performance Testing with JMeter (Basics)

Interview tip: The JD lists JMeter as required. If your hands-on experience is limited, say so, then show that you know the concepts and the building blocks below.

Q312. What is performance testing? What are its types?

Performance testing checks speed, stability and scalability under load.

Type	Purpose
Load	Behaviour under the expected number of users
Stress	Behaviour beyond the limit, to find the breaking point and recovery
Spike	Behaviour on a sudden jump in users
Endurance (soak)	Behaviour under normal load for a long time, to find memory leaks
Volume	Behaviour with a large amount of data
Scalability	How capacity grows as resources are added

Q313. What is JMeter?

An open-source Java tool from Apache for load and performance testing of web applications, APIs, databases and more. It simulates many users with threads and reports response times and throughput.

Q314. What are the main elements of a JMeter test plan?

Element	Role
Test Plan	The container for everything
Thread Group	The users: number of threads, ramp-up period, loop count
Samplers	The requests, such as HTTP Request
Config elements	HTTP Request Defaults, Header Manager, Cookie Manager, CSV Data Set Config
Timers	Think time between requests, such as Constant Timer
Assertions	Checks on the response: Response Assertion, Duration Assertion, JSON Assertion
Pre and post processors	Run before or after a sampler; JSON Extractor and Regular Expression Extractor capture values
Listeners	Results: View Results Tree, Summary Report, Aggregate Report
Logic controllers	Flow: Loop, If, Transaction Controller

Q315. What are threads, ramp-up period and loop count?

Threads are virtual users. Ramp-up is the time JMeter takes to start all of them: 100 threads with a 50-second ramp-up starts 2 users per second. Loop count is how many times each user repeats the test.

Q316. What is correlation? What is parameterization?

Correlation captures a dynamic value from a response, such as a session id or token, with an extractor and passes it to later requests. Parameterization feeds different test data to each user, usually with CSV Data Set Config. Both are the same ideas as request chaining and data-driven testing in Postman.

Q317. Which metrics do you look at?

Average, median, 90th, 95th and 99th percentile response time, throughput (requests per second), error percentage, number of concurrent users, latency, and on the server side CPU, memory and database usage.

Q318. What is the difference between throughput and response time? Between latency and response time?

Throughput is how many requests the server handles per unit of time. Response time is how long one request takes from send to complete response. Latency is the time to the first byte of the response.

Q319. Why do we look at the 90th percentile and not only the average?

The average hides slow requests. The 90th percentile says 90 percent of requests were faster than this value, which reflects what most users really experience.

Q320. How do you run JMeter for a real test?

Build and debug the script in the GUI with a few users, then run in non-GUI mode, because the GUI consumes resources: `jmeter -n -t test.jmx -l results.jtl -e -o report`. For very high load, use distributed testing with several load generators. It can run from Jenkins.

Q321. How would you approach a performance test?

Gather requirements (expected users, target response time, critical transactions), design the workload, prepare scripts and test data, set up an environment close to production, run a baseline, run the load tests, monitor the servers, analyse bottlenecks, report, and retest after tuning.

12. Security Testing (Basics)

Interview tip: The JD asks for API interception, intrusion testing, OWASP Top 10 and ZAP. Learn this vocabulary well. Present it as knowledge of concepts with basic exposure unless you have done it hands-on.

Q322. What is security testing?

Testing that finds weaknesses that could let someone access data or functions they should not. Its goals are confidentiality, integrity, availability, authentication, authorization and non-repudiation.

Q323. What is the difference between vulnerability assessment and penetration testing?

Vulnerability assessment scans a system to list known weaknesses. Penetration testing actively tries to exploit them to prove the real risk.

Q324. What is the OWASP Top 10?

A list published by OWASP of the most critical web application security risks. The 2021 list:

#	Risk	In simple words
A01	Broken Access Control	A user can reach data or actions that are not theirs
A02	Cryptographic Failures	Sensitive data is not encrypted properly
A03	Injection	Untrusted input is run as a command: SQL injection, XSS
A04	Insecure Design	Security flaws in the design itself
A05	Security Misconfiguration	Default passwords, open ports, verbose errors
A06	Vulnerable and Outdated Components	Old libraries with known flaws
A07	Identification and Authentication Failures	Weak login, session and password handling
A08	Software and Data Integrity Failures	Untrusted updates, plugins, pipelines
A09	Security Logging and Monitoring Failures	Attacks go unnoticed
A10	Server-Side Request Forgery (SSRF)	The server is tricked into calling an internal URL

OWASP also publishes a separate API Security Top 10, led by Broken Object Level Authorization.

Q325. What is API interception? Which tools are used?

Placing a proxy between the client and the server to capture, inspect and modify requests and responses in transit. Tools: Burp Suite, OWASP ZAP, Postman's proxy, Charles and Fiddler. The tester changes parameters, headers, tokens and payloads to see whether the server validates them properly.

Q326. What does "intrude" or intruder testing mean?

Sending a captured request many times automatically with different payloads at chosen positions. Burp Suite's Intruder and ZAP's Fuzzer do this. It is used for brute-force checks, fuzzing inputs, enumerating ids, and testing rate limits.

Q327. What security tests would you run on an API?

Area from the JD	What to test	Expected
Authentication	No token, invalid token, expired token, tampered JWT	401
Authorization	User A's token on user B's resource; a normal user calling an admin API	403 or 404
Rate limiting	Many rapid requests, such as repeated login attempts	429, account lock
Data exposure	Passwords, tokens, internal ids or stack traces in the response; more fields than the UI needs	None present
Payload manipulation	Change price, quantity, role or id in the body; add extra fields; send negative values	Rejected by server-side validation
Injection	SQL and script strings in parameters and body	Rejected, never a 500
Transport	HTTP instead of HTTPS; missing security headers	Refused or redirected
Methods	Unsupported methods such as PUT or DELETE where not allowed	405

Q328. What is BOLA or IDOR?

Broken Object Level Authorization, also called Insecure Direct Object Reference. The API checks that you are logged in but not that the object belongs to you. Changing `/orders/101` to `/orders/102` returns another user's order. It is the most common API vulnerability.

Q329. What is SQL injection? How do you test for it?

Input that changes the SQL query the server runs, such as entering `' OR '1'='1` in a login field. A vulnerable system logs you in or returns a database error. The fix is parameterized queries and input validation.

Q330. What is XSS? What is CSRF?

Cross-Site Scripting: an attacker's script is stored or reflected by the site and runs in another user's browser. Test by entering `<script>alert(1)</script>`; it must be shown as text, not executed.
Cross-Site Request Forgery: a logged-in user's browser is tricked into sending an unwanted request; it is prevented with CSRF tokens and same-site cookies.

Q331. What is OWASP ZAP?

Zed Attack Proxy, a free security testing tool. It works as an intercepting proxy, has a spider that crawls the application, a passive scanner that inspects traffic, an active scanner that attacks to find vulnerabilities, and a fuzzer. It produces a report by risk level and can run in a CI pipeline.

Q332. What is the difference between ZAP and Burp Suite?

Both are intercepting proxies with scanners. ZAP is free and open source. Burp Suite has a free community edition and a paid professional edition with a stronger scanner, and is the tool most penetration testers use.

Q333. What is the difference between SAST and DAST?

Static Application Security Testing scans source code without running it. Dynamic Application Security Testing attacks the running application from outside. ZAP is a DAST tool.

Q334. What is the difference between encryption, hashing and encoding?

Encryption is reversible with a key and protects confidentiality. Hashing is one-way and is used for passwords and integrity. Encoding, such as Base64, only changes the format and gives no security.

13. Test Strategy, Scenarios and Behavioural Questions

Test strategy and ownership

Interview tip: The JD says "define and own the overall test strategy". Expect at least one open question such as "how would you set up quality for a new product?" Structure wins here: speak in clear steps.

Q335. How would you define a test strategy for a new product or feature?

- Understand the product: users, critical flows, architecture, integrations, risks.
- Scope: what is in and out; functional and non-functional needs.
- Levels: unit tests by developers, API tests as the main automated layer, a small UI suite for end-to-end flows (the test pyramid).
- Types: functional, regression, integration, performance, security, compatibility.
- Automation approach: tools, framework, what to automate first, coverage targets.
- Environments and test data: how they are created and refreshed.
- Process: entry and exit criteria, defect workflow, severity definitions.
- CI/CD: which suites run at which stage and act as gates.
- Metrics and reporting: coverage, pass rate, leakage, cycle time.
- Risks and mitigation.

Q336. You join a team with no automation. How do you start?

I first learn the product and the manual regression suite, and identify the critical flows. I choose tools that fit the team's skills and the application. I build a small, clean framework and automate the smoke suite first, to show value quickly. I put it into CI from day one. Then I grow coverage by priority: API tests for business logic, UI tests for key journeys. I set coding standards and reviews, and track coverage and run time.

Q337. How do you decide whether a release is ready?

I check the exit criteria: all planned tests run, regression green, no open critical or high defects, known issues documented and accepted, performance and security checks done where required, and the rollback plan in place. I present the facts and the remaining risk; the decision is shared with the product owner.

Q338. How do you estimate testing effort?

I break the work into tasks: analysis, test design, data and environment setup, execution, automation, regression, defect retesting. I estimate each from complexity and past experience, add a buffer for risk, and refine it in planning with the team.

Q339. How do you prioritise when everything is urgent?

By risk and business impact: what breaks the business if it fails, what changed, and what has failed before. I agree the order with the product owner and make the trade-offs visible.

Q340. How do you manage test data?

Tests create the data they need, through APIs or database scripts, and clean up afterwards. Static reference data is kept in files. I avoid sharing data between tests and never use real customer data without masking.

Q341. What is shift-left testing?

Moving testing earlier: reviewing requirements, testing APIs before the UI exists, running automated tests on every commit, and working with developers during design. Defects are found sooner and cost less.

Scenario questions

Q342. How would you test an e-commerce checkout?

- Functional: add to cart, update quantity, remove, apply coupon, address, shipping options, each payment method, order confirmation, email.
- Negative: invalid card, expired coupon, out-of-stock item during checkout, payment timeout, double click on Pay, back button after payment.
- API: cart, price calculation, order and payment endpoints, with the total checked on the server, never trusted from the client.
- Non-functional: load on sale days, security of payment data, behaviour across browsers and devices.

Q343. The same API works in Postman but fails from the application. How do you debug?

I compare the two requests: URL and environment, method, headers (especially Content-Type and Authorization), body format, and cookies. I capture the application's real request from the browser's Network tab or a proxy and replay it in Postman. Differences usually sit in headers, tokens, encoding, or CORS, which affects browsers and not Postman.

Q344. A page takes 10 seconds to load. How do you investigate?

I check the browser's Network tab to see which call is slow: a specific API, a large image or script, or many requests. I test that API alone in Postman to separate front-end from back-end. I check whether it happens for all data or only large data sets, and under load or always, then report with timings.

Q345. How do you test when the back-end API is not ready?

I work from the API contract (Swagger), prepare test cases and collections, and use a mock server that returns the agreed responses, so the tests are ready to run the day the API is delivered.

Q346. You found a critical bug an hour before release. What do you do?

I reproduce it, gather evidence, and inform the lead and product owner immediately with the impact and how often it occurs. I give the options: fix and retest, release with a workaround or feature flag, or delay. I do not decide alone and I do not hide it.

Q347. How do you make sure a UI test is not testing something the API should test?

I ask what the test proves. Business rules, calculations and validations belong at the API layer, where they run fast and reliably. UI tests cover what only the UI can show: the user journey, rendering, and integration of the front end with the back end.

Behavioural questions

Use the STAR method: Situation, Task, Action, Result. Prepare one real story for each question and keep it under two minutes.

Q348. Tell me about a challenging bug you found.

Pick a real one. Describe what made it hard to find (only with specific data, only under timing, only in one environment), how you isolated it, and its impact.

Q349. Tell me about a time you disagreed with a developer.

Show that you stayed with facts, listened, reproduced together, and used the requirement or the product owner to settle it. End with the relationship intact.

Q350. Tell me about a time you missed a bug.

Be honest. Describe it briefly, then spend most of the answer on what you did after: root cause, the test you added, and the process change.

Q351. Tell me about something you automated or improved on your own initiative.

Give numbers: time saved, tests added, failures reduced.

Q352. What are your strengths and weaknesses?

Strengths with proof, such as attention to detail shown by a bug you caught. A real weakness with what you are doing about it, for example: "My performance testing experience is limited, so I am learning JMeter and practising on sample APIs."

Q353. Where do you see yourself in a few years?

Growing into a senior SDET who owns the quality strategy across UI, API, performance and security, and mentors others.

SQL basics (often asked to SDETs)

Q354. Which SQL do you use in testing?

To verify that data shown in the UI or returned by an API matches the database, and to set up or clean test data.

```
SELECT name, email FROM users WHERE status = 'ACTIVE' ORDER BY created_at DESC;

SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;

-- second highest salary
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

-- duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

Q355. Explain the joins, and the difference between WHERE and HAVING, DELETE, TRUNCATE and DROP.

INNER JOIN returns matching rows from both tables. LEFT JOIN returns all rows from the left table plus matches. RIGHT JOIN is the reverse. FULL JOIN returns all rows from both. WHERE filters rows before grouping; HAVING filters groups after GROUP BY. DELETE removes chosen rows and can be rolled back; TRUNCATE removes all rows quickly; DROP removes the table itself.

Q356. What are primary and foreign keys?

A primary key uniquely identifies each row and cannot be null. A foreign key is a column that refers to the primary key of another table and links the two.

Questions to ask the interviewer

- What does the current automation stack look like, and how much is Selenium versus Playwright?
- How is testing split between the QA team and developers?
- How are performance and security testing done today, and how would this role contribute?
- What would success look like in the first three months?

14. Last-Day Revision Sheet

Say these without thinking

Topic	The one line
Severity vs priority	Severity is impact; priority is urgency
Smoke vs sanity	Smoke: is the build stable, wide and shallow. Sanity: does this fix work, narrow and deep
Retesting vs regression	Retest the fix; regress around it
Verification vs validation	Building it right vs building the right thing
<code>==</code> vs <code>equals()</code>	Reference vs content
Overloading vs overriding	Same class, different parameters, compile time vs parent-child, same signature, runtime
Abstract class vs interface	Shared code and state vs a contract; extend one, implement many
String vs StringBuilder vs StringBuffer	Immutable vs mutable fast vs mutable thread-safe
ArrayList vs LinkedList	Fast read vs fast insert and delete
HashMap vs Hashtable	Not synchronized, one null key vs synchronized, no nulls
<code>final</code> , <code>finally</code> , <code>finalize</code>	Keyword, block, deprecated GC method
<code>throw</code> vs <code>throws</code>	Throw one exception vs declare in the signature
<code>findElement</code> vs <code>findElements</code>	Exception vs empty list
<code>close()</code> vs <code>quit()</code>	Current window vs whole session
Implicit vs explicit wait	Global presence vs one element, one condition
POM vs Page Factory	Pattern vs Selenium's implementation with <code>@FindBy</code>
Hard vs soft assert	Stops vs continues until <code>assertAll()</code>
<code>@Parameters</code> vs <code>@DataProvider</code>	Config from XML vs many data sets from code
Scenario vs Scenario Outline	Runs once vs once per Examples row
Background vs hooks	Gherkin steps before each scenario vs code before and after
PUT vs PATCH	Whole resource vs part
401 vs 403	Not identified vs not permitted
Path vs query parameter	Identify one vs filter a list
<code>git fetch</code> vs <code>git pull</code>	Download vs download and merge
CI vs CD	Build and test on every merge vs always releasable or auto-deployed
Load vs stress	Expected load vs beyond the limit
Authentication vs authorization	Who you are vs what you may do

Before the interview

- ☐ Practise your introduction and your framework explanation aloud, three times each.
- ☐ Write five programs by hand: reverse a string, palindrome, character count, second largest, duplicates in an array.
- ☐ Explain the four OOP pillars with framework examples.

- ☐ Write the window-handling code and one explicit wait from memory.
- ☐ Revise the Selenium exceptions table.
- ☐ Open your Postman collection and your GitHub repository; be ready to share your screen.
- ☐ Revise the status codes and the PUT, PATCH, POST differences.
- ☐ Read the OWASP Top 10 and JMeter element tables once.
- ☐ Prepare three STAR stories and two questions for the interviewer.

During the interview

- Listen to the full question. Take two seconds before you answer.
- Definition, then project example. Keep answers to about a minute unless asked for more.
- When coding, talk through your approach and test with one example at the end.
- If you do not know, say so calmly and offer what you do know.
- Be honest about JMeter and security depth, and show eagerness to own them.

All the best, Ujjwal.